Equation of Motion Time Discretization

Newton's 2nd Law as a first-order system

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{v} \\ \mathbf{M}\dot{\mathbf{v}} = \mathbf{f}(\mathbf{x}, \mathbf{v}) \end{cases}$$

Equation of Motion Time Discretization

Newton's 2nd Law as a first-order system

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{v} \\ \mathbf{M}\dot{\mathbf{v}} = \mathbf{f}(\mathbf{x}, \mathbf{v}) \end{cases}$$

Time Discretization

$$\dot{\mathbf{q}} \approx \frac{\mathbf{q}^+ - \mathbf{q}^-}{h}$$

RHS at *t*

Explicit Euler

$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h\,\mathbf{v}^- \\ \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = h\,\mathbf{f}(\mathbf{x}^-, \mathbf{v}^-) \end{cases}$$

Only known quantities on the right: a direct, cheap update. Conditionally stable: stiff materials need tiny time step.

RHS at *t+1*

Implicit Euler

$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h\,\mathbf{v}^+ \\ \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = h\,\mathbf{f}(\mathbf{x}^+, \mathbf{v}^+) \end{cases}$$

Unknowns on both sides: a nonlinear system to solve every step. Unconditionally stable, even for large time steps and stiff materials.

Equation of Motion Time Discretization

Newton's 2nd Law as a first-order system

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{v} \\ \mathbf{M}\dot{\mathbf{v}} = \mathbf{f}(\mathbf{x}, \mathbf{v}) \end{cases}$$

Time Discretization

$$\dot{\mathbf{q}} \approx \frac{\mathbf{q}^+ - \mathbf{q}^-}{h}$$

RHS at t

Explicit Euler

$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h\,\mathbf{v}^- \\ \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = h\,\mathbf{f}(\mathbf{x}^-, \mathbf{v}^-) \end{cases}$$

Only known quantities on the right: a direct, cheap update. Conditionally stable: stiff materials need tiny time step.

RHS at $t+1$

Implicit Euler

$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h\,\mathbf{v}^+ \\ \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = h\,\mathbf{f}(\mathbf{x}^+, \mathbf{v}^+) \end{cases}$$

Unknowns on both sides: a nonlinear system to solve every step. Unconditionally stable, even for large time steps and stiff materials.

Equation of Motion Time Discretization

Newton's 2nd Law as a first-order system

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{v} \\ \mathbf{M}\dot{\mathbf{v}} = \mathbf{f}(\mathbf{x}, \mathbf{v}) \end{cases}$$

Time Discretization

$$\dot{\mathbf{q}} \approx \frac{\mathbf{q}^+ - \mathbf{q}^-}{h}$$

RHS at *t*

Explicit Euler

$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h\,\mathbf{v}^- \\ \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = h\,\mathbf{f}(\mathbf{x}^-, \mathbf{v}^-) \end{cases}$$

Only known quantities on the right: a direct, cheap update. Conditionally stable: stiff materials need tiny time step.

RHS at *t+1*

Implicit Euler

$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h\,\mathbf{v}^+ \\ \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = h\,\mathbf{f}(\mathbf{x}^+, \mathbf{v}^+) \end{cases}$$

Unknowns on both sides: a nonlinear system to solve every step. Unconditionally stable, even for large time steps and stiff materials.

Equation of Motion Time Discretization

Newton's 2nd Law as a first-order system

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{v} \\ \mathbf{M}\dot{\mathbf{v}} = \mathbf{f}(\mathbf{x}, \mathbf{v}) \end{cases}$$

Time Discretization

$$\dot{\mathbf{q}} \approx \frac{\mathbf{q}^+ - \mathbf{q}^-}{h}$$

RHS at t

Explicit Euler

$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h\,\mathbf{v}^- \\ \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = h\,\mathbf{f}(\mathbf{x}^-, \mathbf{v}^-) \end{cases}$$

Only known quantities on the right: a direct, cheap update. Conditionally stable: stiff materials need tiny time step.

RHS at $t+1$

Implicit Euler

$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h\,\mathbf{v}^+ \\ \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = h\,\mathbf{f}(\mathbf{x}^+, \mathbf{v}^+) \end{cases}$$

Unknowns on both sides: a nonlinear system to solve every step. Unconditionally stable, even for large time steps and stiff materials.

Implicit Euler

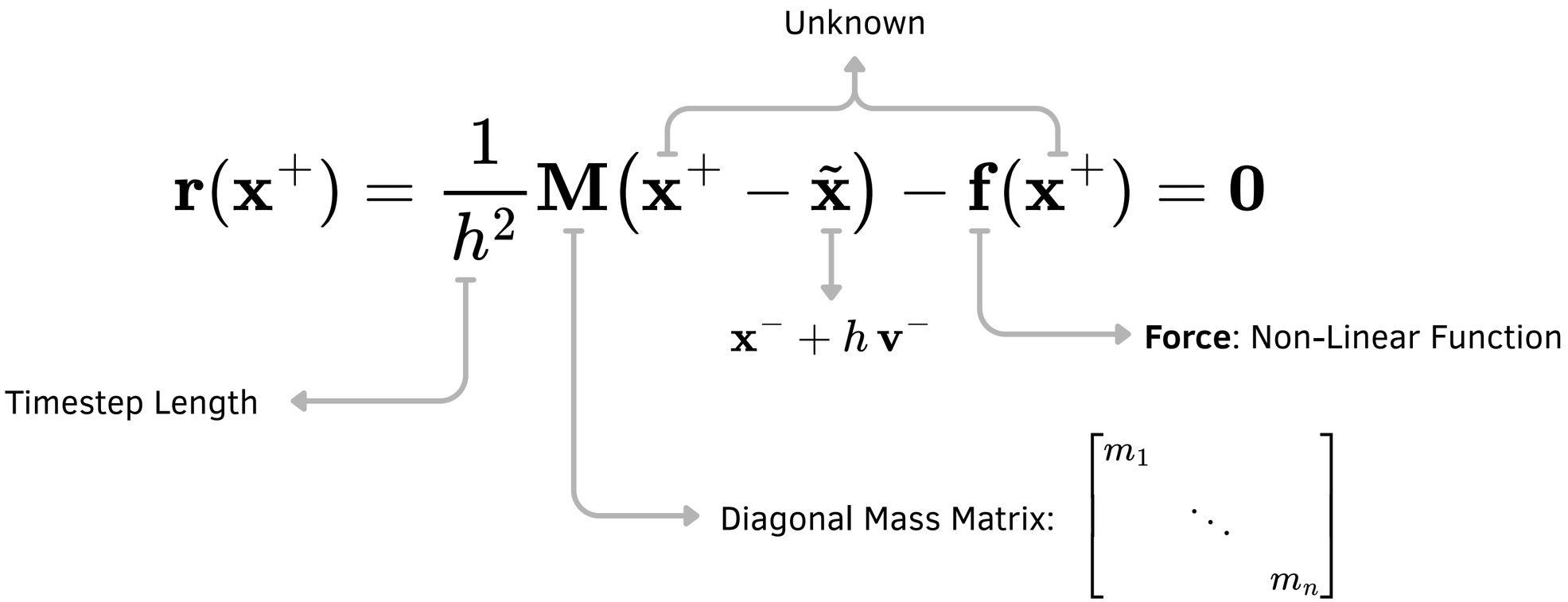
$$\begin{cases} \mathbf{x}^+ = \mathbf{x}^- + h\,\mathbf{v}^+ \\ \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) = h\,\mathbf{f}(\mathbf{x}^+, \mathbf{v}^+) \end{cases}$$

$$\mathbf{r}(\mathbf{x}^+) = \frac{1}{h^2}\mathbf{M}\left(\mathbf{x}^+ - \tilde{\mathbf{x}}\right) - \mathbf{f}(\mathbf{x}^+) = \mathbf{0}$$

Non-Linear System of Equations

$$\frac{1}{h^2}\begin{bmatrix} m_1 & & & \\ & m_2 & & \\ & & \ddots & \\ & & & m_n \end{bmatrix}\begin{bmatrix} \mathbf{x}_1^+ - \tilde{\mathbf{x}}_1 \\ \mathbf{x}_2^+ - \tilde{\mathbf{x}}_2 \\ \vdots \\ \mathbf{x}_n^+ - \tilde{\mathbf{x}}_n \end{bmatrix} - \begin{bmatrix} \mathbf{f}_1(\mathbf{x}^+) \\ \mathbf{f}_2(\mathbf{x}^+) \\ \vdots \\ \mathbf{f}_n(\mathbf{x}^+) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Implicit Euler



Newton-Raphson Method

We have:

$$\mathbf{g} : \mathbb{R}^n \to \mathbb{R}^n$$

We want:

$$\mathbf{g}(\mathbf{z}^*) = \mathbf{0}$$

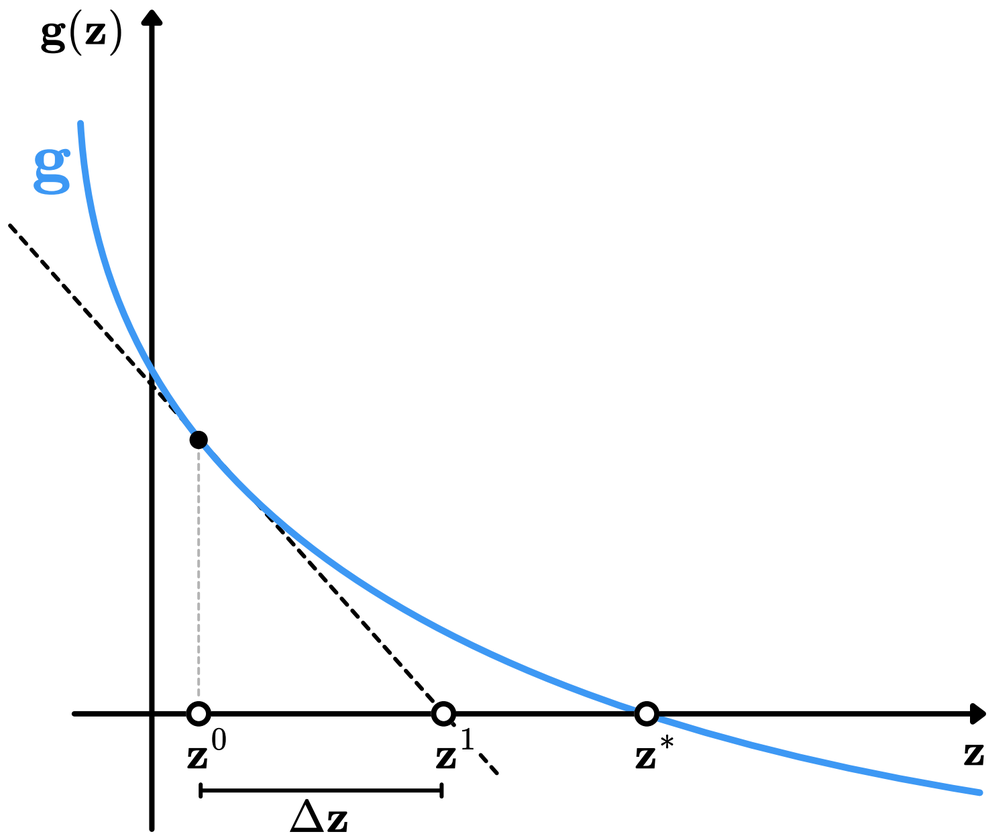
Linearize around a guess:

$$\mathbf{g}(\mathbf{z}^{(k)} + \Delta\mathbf{z}) \approx \mathbf{g}(\mathbf{z}^{(k)}) + \frac{\partial \mathbf{g}}{\partial \mathbf{z}}(\mathbf{z}^{(k)})\Delta\mathbf{z} = \mathbf{0}$$

Linear System of Equations

$$\frac{\partial \mathbf{g}}{\partial \mathbf{z}}\Delta\mathbf{z} = -\mathbf{g}(\mathbf{z}^{(k)})$$

$$\mathbf{z}^{(k+1)} = \mathbf{z}^{(k)} + \Delta\mathbf{z}$$



Newton-Raphson Method for Implicit Dynamic System

We have:

$$\mathbf{r} : \mathbb{R}^{3n} \to \mathbb{R}^{3n} \quad \text{such that:} \quad \mathbf{r}(\mathbf{x}) = \frac{1}{h^2}\mathbf{M}\left(\mathbf{x} - \tilde{\mathbf{x}}\right) - \mathbf{f}(\mathbf{x})$$

We want:

$$\mathbf{r}\left(\mathbf{x}^+\right) = \mathbf{0}$$

Linearize around a guess:

$$\frac{\partial \mathbf{r}}{\partial \mathbf{x}} = \underbrace{\frac{1}{h^2}\mathbf{M} - \frac{\partial \mathbf{f}}{\partial \mathbf{x}}}_{\mathbf{A} \in \mathbb{R}^{3n \times 3n}}$$

Re-built each iteration: very expensive

$$\mathbf{A}\left(\mathbf{x}^{(k)}\right) \Delta\mathbf{x} = -\mathbf{r}\left(\mathbf{x}^{(k)}\right)$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta\mathbf{x}$$

Newton-Raphson Method Single Iteration

Expanded:

$$\underbrace{\left[\mathbf{M} - h^2 \frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right]}_{\mathbf{A}(\mathbf{x}^-)} \Delta\mathbf{x} = \underbrace{h^2 \mathbf{f}(\mathbf{x}^-) + h\,\mathbf{M}\mathbf{v}^-}_{\mathbf{b}(\mathbf{x}^-)}$$

Algorithm:

1. $\texttt{build: } \mathbf{A}, \mathbf{b}$
2. $\texttt{solve: } \mathbf{A}\,\Delta\mathbf{x} = \mathbf{b}$
3. $\mathbf{x}^+ = \mathbf{x}^- + \Delta\mathbf{x}$
4. $\mathbf{v}^+ = \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$

Recasting Root-Finding as Minimization

Does exists a: $g(\mathbf{x})$ such that: $\nabla g(\mathbf{x}) = \mathbf{r}(\mathbf{x})$

$$g(\mathbf{x}) = \frac{1}{2h^2}(\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + U(\mathbf{x})$$

$$\mathbf{f}(\mathbf{x}) = -\frac{\partial U}{\partial \mathbf{x}}(\mathbf{x})$$ Conservative Forces

$$\nabla g(\mathbf{x}) = \frac{1}{h^2}\mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) - \mathbf{f}(\mathbf{x}) = \mathbf{r}(\mathbf{x})$$

Recasting Root-Finding as Minimization

Does exists a: $g(\mathbf{x})$ such that: $\nabla g(\mathbf{x}) = \mathbf{r}(\mathbf{x})$

$$g(\mathbf{x}) = \frac{1}{2h^2}(\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + U(\mathbf{x})$$

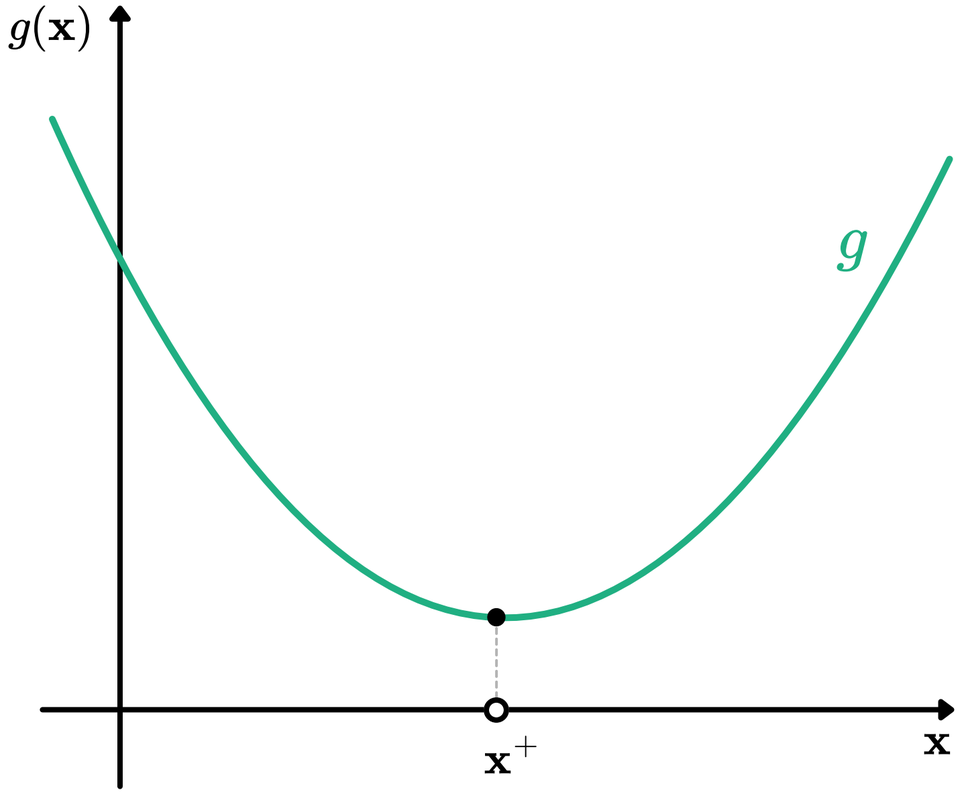
$$\mathbf{f}(\mathbf{x}) = -\frac{\partial U}{\partial \mathbf{x}}(\mathbf{x})$$ Conservative Forces

$$\mathbf{r}(\mathbf{x}) = \mathbf{0} \iff \nabla g(\mathbf{x}) = \mathbf{0} \overset{\text{if } \textbf{Convex}}{\iff} \min_{\mathbf{x}} g(\mathbf{x})$$

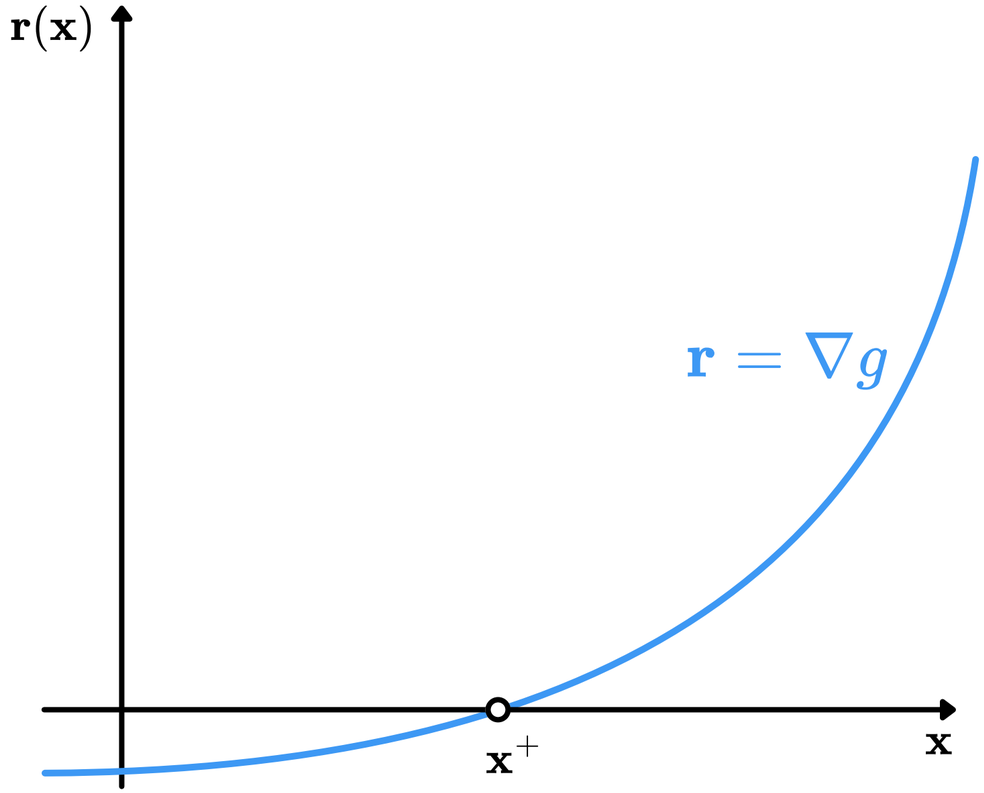
Root-Finding and Minimization Equivalence

Equivalent Problems:

$$\mathbf{x}^+ = \arg\min_{\mathbf{x}} g(\mathbf{x})$$



$$\mathbf{r}(\mathbf{x}^+) = 0$$



Descent Methods How to Solve the Minimization Problem

1 $\texttt{Initialize } \mathbf{x}^{(0)}$
2 **for** $k = 0, 1, \ldots, N-1$ **do**
3 $\quad \Delta\mathbf{x} = \texttt{DescentDirection()}$
4 $\quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \cdot \Delta\mathbf{x}$
5 $\mathbf{x}^{+} = \mathbf{x}^{(N)}$
6 $\mathbf{v}^{+} = \frac{1}{h}(\mathbf{x}^{+} - \mathbf{x}^{-})$

Jacobian:

$$\mathbf{J} = \nabla g(\mathbf{x}) = \mathbf{r}(\mathbf{x}) \qquad \in \mathbb{R}^{3n}$$

Hessian:

$$\mathbf{H} = \nabla^2 g(\mathbf{x}) = \mathbf{A}(\mathbf{x}) \quad \in \mathbb{R}^{3n \times 3n}$$

Gradient Descent:

$$\Delta\mathbf{x}^{(k)} = -\mathbf{J}^{(k)}$$

Newton:

$$\Delta\mathbf{x}^{(k)} = -\left[\mathbf{H}^{(k)}\right]^{-1}\mathbf{J}^{(k)}$$

Quasi-Newton:

$$\Delta\mathbf{x}^{(k)} = -\mathbf{P}^{(k)}\mathbf{J}^{(k)}, \quad \mathbf{P}^{(k)} \approx \left[\mathbf{H}^{(k)}\right]^{-1}$$

Projective Dynamics:

$$\Delta\mathbf{x}^{(k)} = -\mathbf{L}_{\mathrm{PD}}^{-1}\,\mathbf{J}_{\mathrm{PD}}^{(k)}$$

$$\mathbf{J}_{\mathrm{PD}} = \nabla g_{\mathrm{PD}}(\mathbf{x}), \quad \mathbf{L}_{\mathrm{PD}} \approx \mathbf{H}\ (\text{fixed})$$

Baraff-Witkin:

$$\Delta\mathbf{x} = -\mathbf{H}^{-1}\mathbf{J}, \qquad k = 1$$

Newton’s Method

Newton's Minimization Method = Newton-Raphson Root Finding's Method

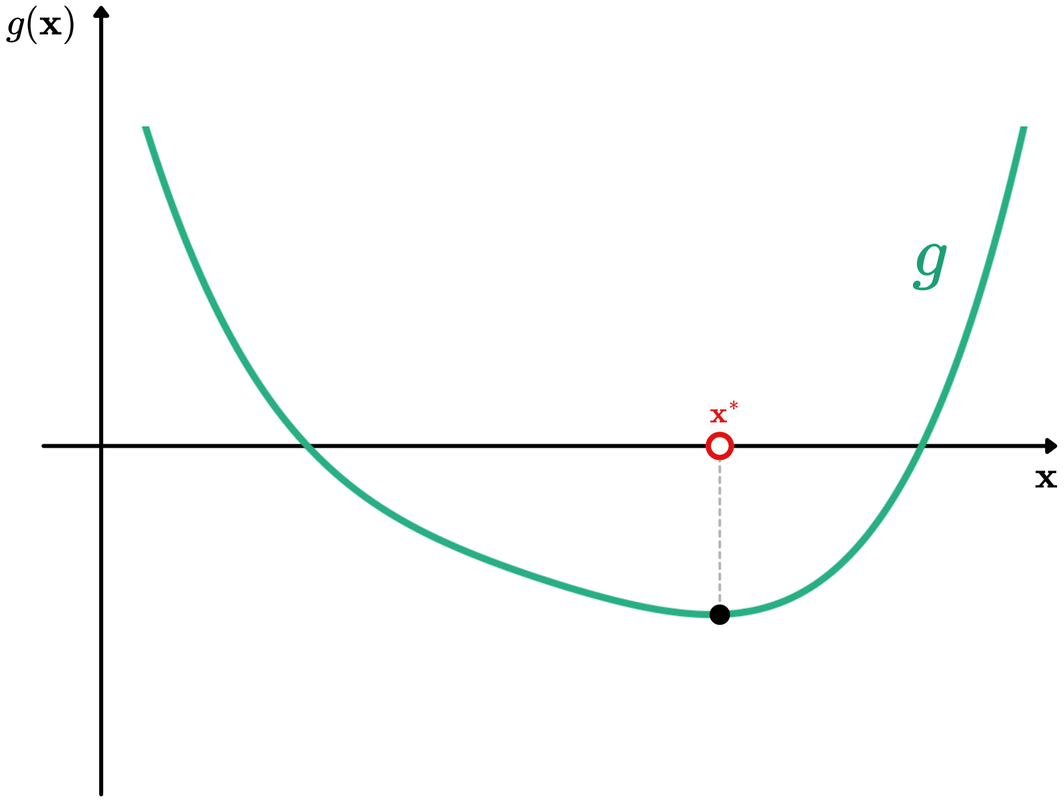
function NEWTONMETHOD($\mathbf{x}^-, \mathbf{v}^-, N$)

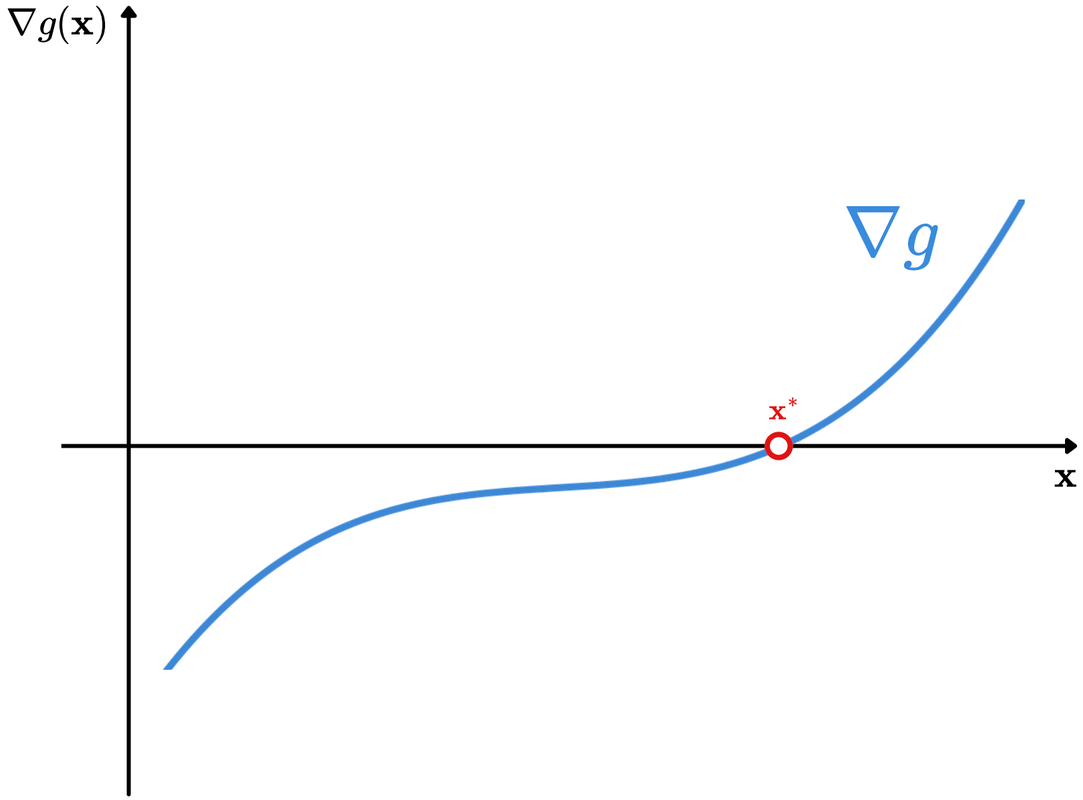
1. Initialize $\mathbf{x}^{(0)}$
2. **for** $k = 0, 1, \ldots, N-1$ **do**
3. BUILD $\mathbf{J}^{(k)}$, $\mathbf{H}^{(k)}$
4. SOLVE $\mathbf{H}^{(k)} \Delta\mathbf{x} = -\mathbf{J}^{(k)}$
5. $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \Delta\mathbf{x}$
6. $\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$
7. $\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$

function NEWTONRAPHSON($\mathbf{x}^-, \mathbf{v}^-, N$)

1. Initialize $\mathbf{x}^{(0)}$
2. **for** $k = 0, 1, \ldots, N-1$ **do**
3. BUILD $\mathbf{A}^{(k)}$, $\mathbf{b}^{(k)}$
4. SOLVE $\mathbf{A}^{(k)} \Delta\mathbf{x} = \mathbf{b}^{(k)}$
5. $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \Delta\mathbf{x}$
6. $\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$
7. $\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$

Convexity and Newton's Method When the Energy **IS** Convex

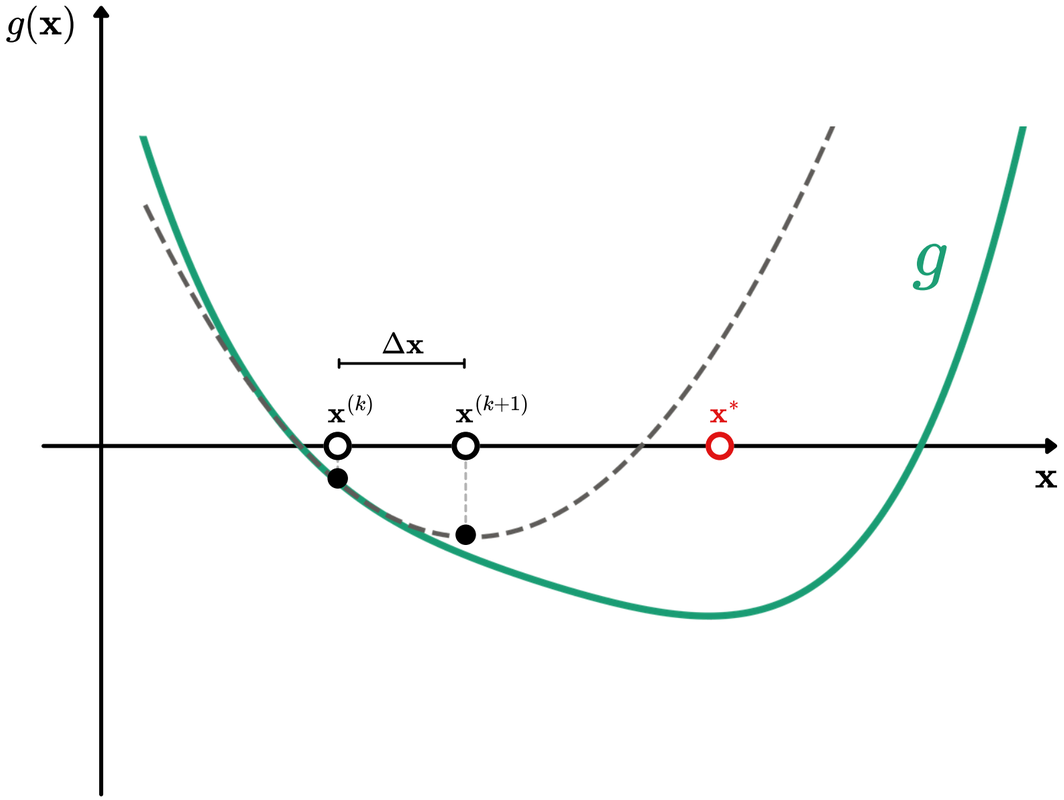




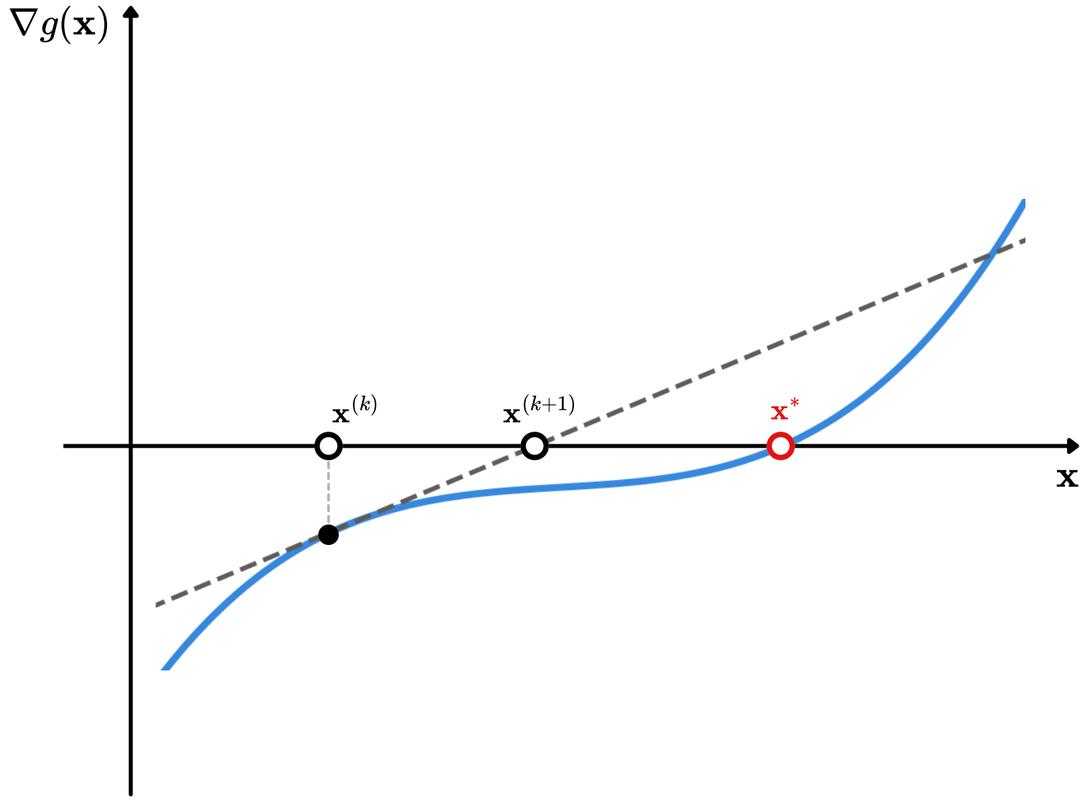
Convexity and Newton's Method When the Energy **IS** Convex

Minimize the local quadratic model of the function g

$$g(\mathbf{x}^{(k+1)}) < g(\mathbf{x}^{(k)})$$

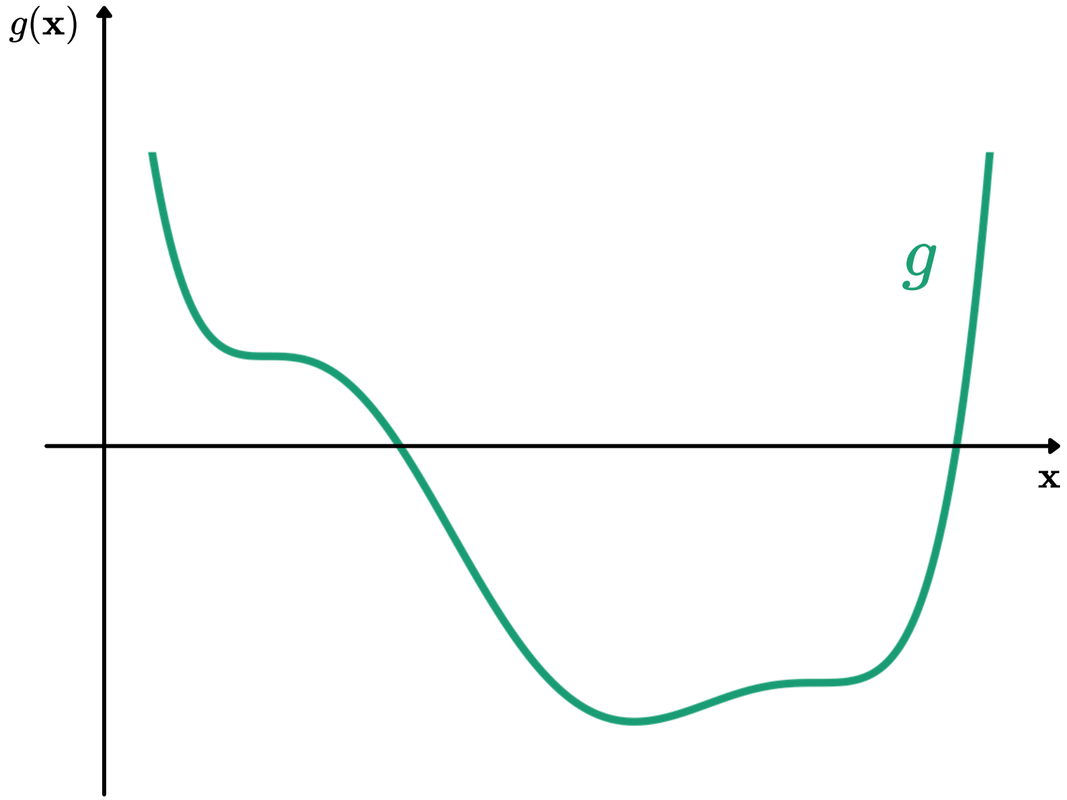


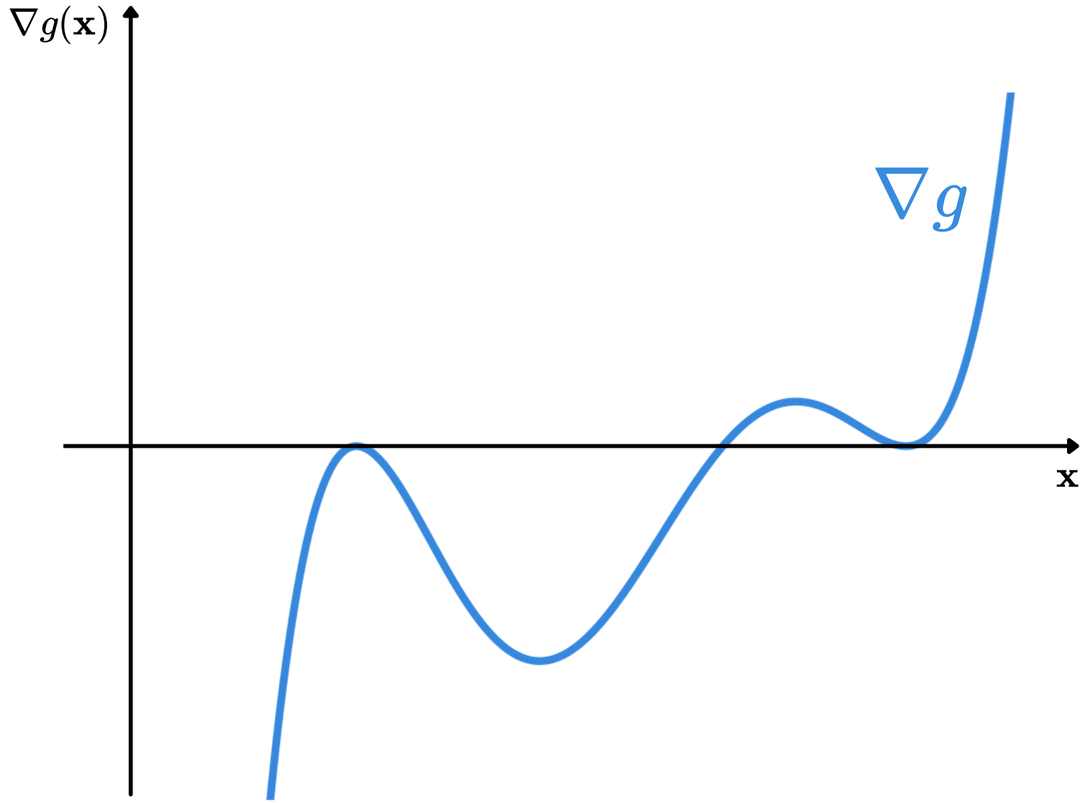
Find the root of the tangent ∇g



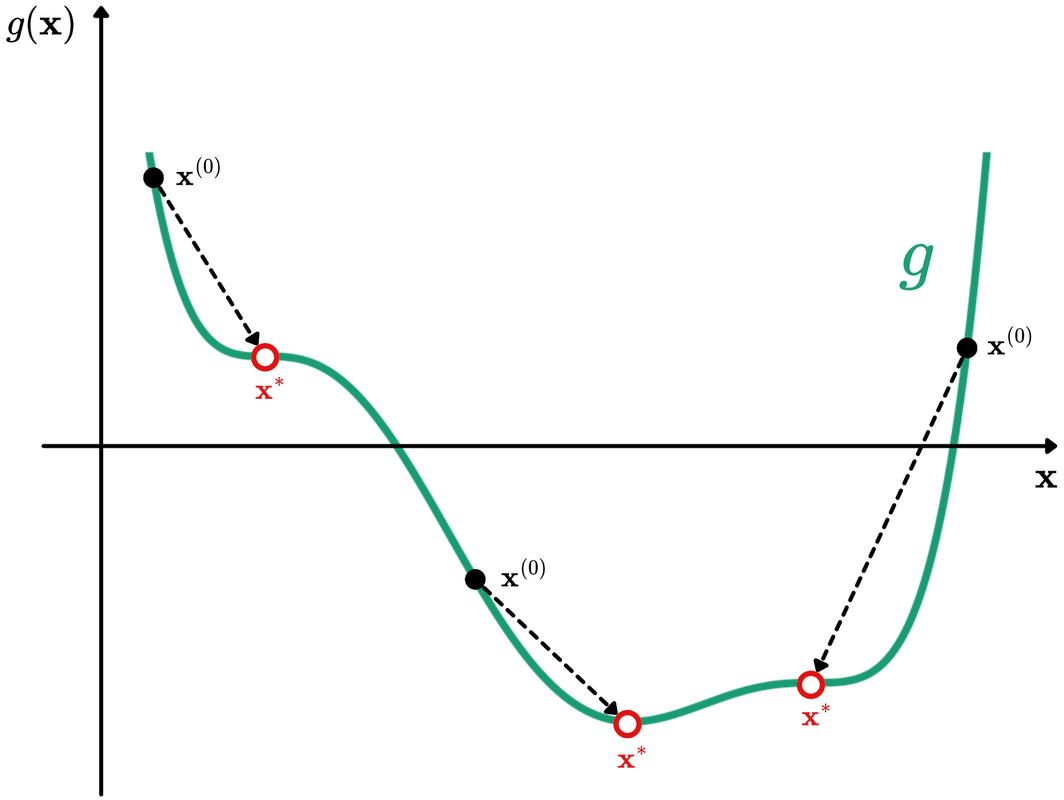
Convexity and Newton's Method When the Energy **IS NOT** Convex

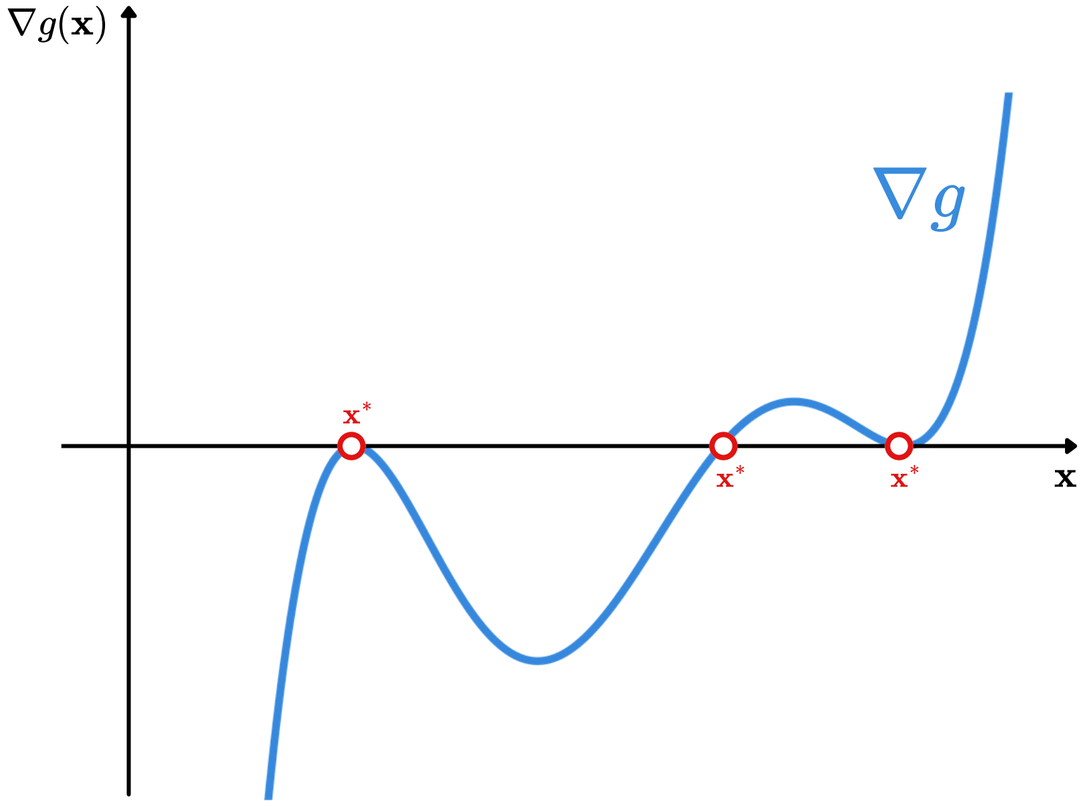
- There might be multiple stationary points
- There might be areas where the curvature is negative





Convexity and Newton's Method: Multiple Stationary Points

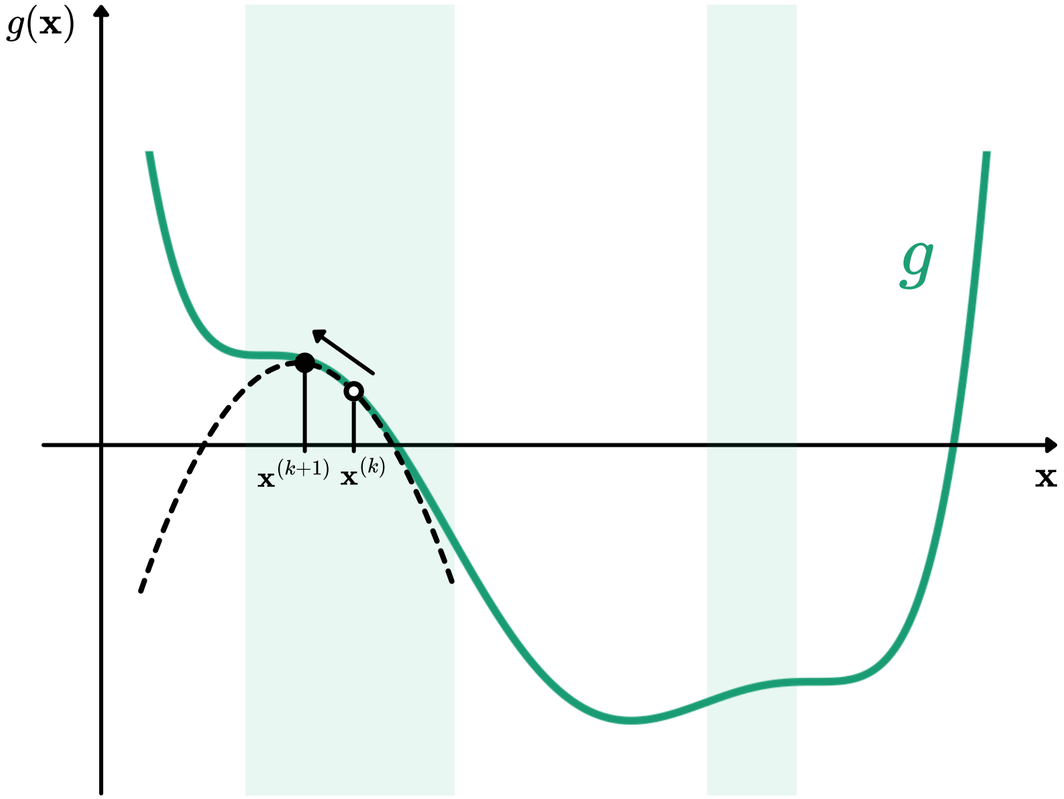


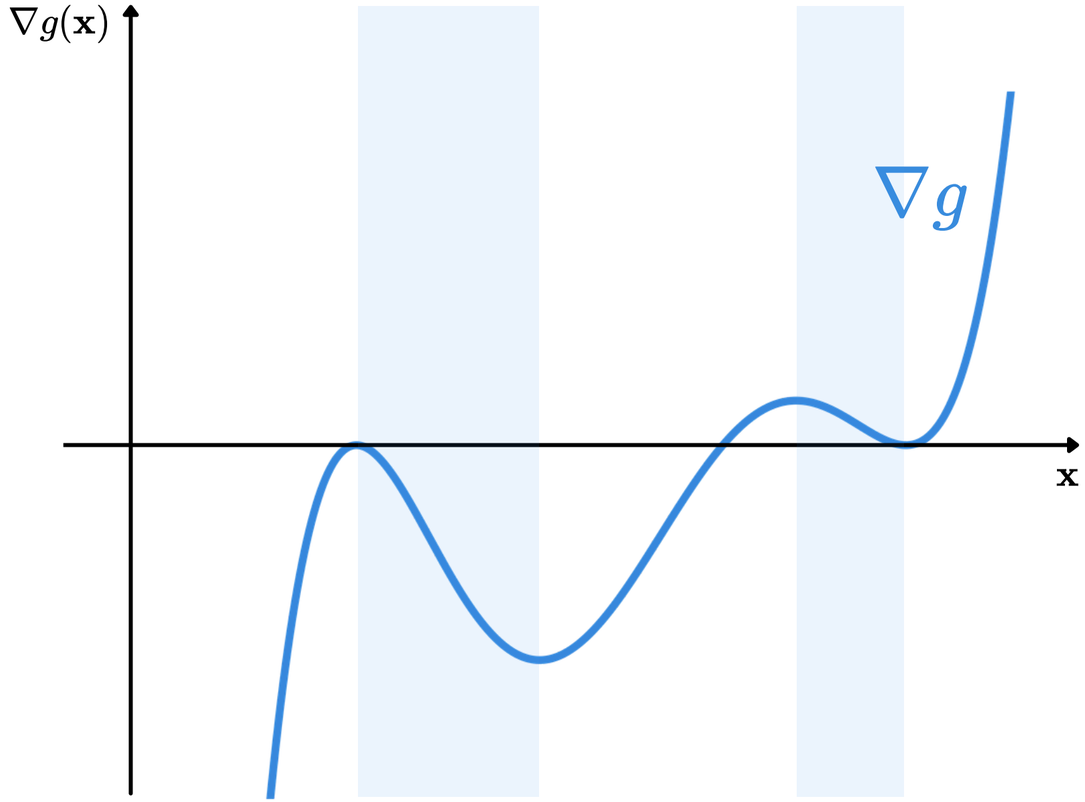


Convexity and Newton's Method: Negative Curvature

If the curvature is negative, by taking the Newton step, there might be an **increase** in energy.
The quadratic model has no minimum, so Newton jumps to its maximum.

$$g(\mathbf{x}^{(k+1)}) > g(\mathbf{x}^{(k)})$$





Convexity and Newton's Method: Solve the Negative Curvature Problem

A matrix is SPD (**Symmetric Positive Definite**), written $\mathbf{A} \succ 0$ if it is symmetric and all its eigenvalues are positive:

$$\mathbf{A} = \mathbf{A}^\top \qquad \text{and} \qquad \mathbf{A} = \mathbf{Q}\,\mathbf{\Lambda}\,\mathbf{Q}^\top, \quad \lambda_i > 0 \quad \forall\, i.$$

- $\mathbf{A} \succ 0,\ \mathbf{B} \succ 0 \ \Rightarrow\ \mathbf{A} + \mathbf{B} \succ 0$
- $\mathbf{A} \succ 0,\ \mathbf{B} \not\succ 0 \ \Rightarrow\ \mathbf{A} + \mathbf{B}$ no guarantee

$\mathbf{H} \not\succ 0:$ $\quad \Delta\mathbf{x} = -\mathbf{H}^{-1}\mathbf{J}$ then $\Delta\mathbf{x}$ may not be a descent direction

$\bar{\mathbf{H}} \succ 0:$ $\quad \Delta\mathbf{x} = -\bar{\mathbf{H}}^{-1}\mathbf{J}$ then $\alpha \cdot \Delta\mathbf{x}$ is guaranteed to be a descent direction for a sufficiently small $\alpha > 0$

$\bar{\mathbf{H}}$ is the SPD-Projection of $\mathbf{H}$

Convexity and Newton's Method: Solve the Negative Curvature Problem

How to fix the Energy's Hessian

Always Convex

$$g(\mathbf{x}) = \frac{1}{2h^2}(\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + U(\mathbf{x})$$

$$\mathbf{H} = \frac{1}{h^2}\mathbf{M} + \nabla^2 U(\mathbf{x})$$

Always SPD

Might not be SPD:
we must **SPD-Project**

$\nabla^2 U(\mathbf{x}) \in \mathbb{R}^{3n \times 3n}$: it is very expensive to SPD-Project such a big matrix

Convexity and Newton's Method: Solve the Negative Curvature Problem

How is the Potential Energy Factor Constructed

$$U(\mathbf{x}) = \sum_e U_e(\mathbf{x}_e)$$

The total energy is the sum of single element energies. Each element (edge, triangle, tethra) involves only a few particles (2, 3 or 4).

$$\nabla^2 U(\mathbf{x}) = \sum_e \nabla^2 U_e(\mathbf{x}_e)$$

$$\nabla^2 U_e(\mathbf{x}_e) \in \mathbb{R}^{6\times 6 | 9\times 9 | 12\times 12}$$

So the Hessian is also a sum of small element Hessians. Each one can be easily SPD-projected independently and in parallel, then assembled into the full Hessian.

Convexity and Newton's Method: Solve the Negative Curvature Problem

How an SPD-Projection is computed

How do we SPD-Project the single element energies:

$$\nabla^2 U_e(\mathbf{x}_e) \in \mathbb{R}^{6\times 6 | 9\times 9 | 12\times 12}$$

Two ways:

- **Numerical:**

$$\nabla^2 U_e = \mathbf{Q}\,\mathbf{\Lambda}\,\mathbf{Q}^\top \longrightarrow \bar{\lambda}_i = \max(\lambda_i, 0) \longrightarrow \overline{\nabla^2 U_e} = \mathbf{Q}\,\bar{\mathbf{\Lambda}}\,\mathbf{Q}^\top$$

- **Analytical:**

  for many frequently used energies the eigenvalues and eigenvectors are known in closed form. This route is usually faster and more robust.

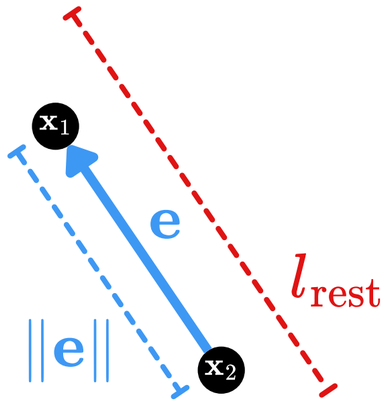
Analytical SPD Projection Quadratic Spring Potential

Spring:

$$\mathbf{x}_e = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{bmatrix} \in \mathbb{R}^6 \qquad \begin{aligned} \mathbf{e} &= \mathbf{x}_1 - \mathbf{x}_2 \\ \hat{\mathbf{e}} &= \frac{\mathbf{e}}{\|\mathbf{e}\|} \end{aligned}$$

Energy:

$$U_e(\mathbf{x}_e) = \frac{k}{2}\left(\|\mathbf{e}\| - l_{\text{rest}}\right)^2$$



Hessian:

$$\mathbf{H}_e = \begin{bmatrix} \mathbf{K} & -\mathbf{K} \\ -\mathbf{K} & \mathbf{K} \end{bmatrix} \in \mathbb{R}^{6\times 6}$$

$$\mathbf{K} = k\left[\hat{\mathbf{e}}\hat{\mathbf{e}}^\top + \left(1 - \frac{l}{\|\mathbf{e}\|}\right)\left(\mathbf{I} - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top\right)\right]$$

Eigenvalues:

λ_1	λ_2	λ_3	λ_4	$\lambda_5 = \lambda_6$
$0,$	$0,$	$0,$	$2k,$	$2k\left(1 - \frac{l}{\|\mathbf{e}\|}\right)$

Can be negative

SPD-Hessian:

$$\bar{\mathbf{H}}_e = \begin{bmatrix} \bar{\mathbf{K}} & -\bar{\mathbf{K}} \\ -\bar{\mathbf{K}} & \bar{\mathbf{K}} \end{bmatrix} \in \mathbb{R}^{6\times 6} \qquad \bar{\mathbf{H}}_e \succeq 0$$

$$\bar{\mathbf{K}} = k\left[\hat{\mathbf{e}}\hat{\mathbf{e}}^\top + \max\left(1 - \frac{l}{\|\mathbf{e}\|},\ 0\right)\left(\mathbf{I} - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top\right)\right]$$

SPD-Aware Newton's Method Algorithm

function SPDAwareNewtonMethod($\mathbf{x}^-, \mathbf{v}^-$)

Initialize $\mathbf{x}^{(0)}$

for $k = 0, 1, \ldots, N-1$ **do**

$\mathbf{J} \leftarrow \nabla g(\mathbf{x}^{(k)})$

$\bar{\mathbf{H}} \leftarrow \text{SPDProjection}\big(\nabla^2 g(\mathbf{x}^{(k)})\big)$

Solve $\quad \bar{\mathbf{H}}\,\Delta\mathbf{x} = -\mathbf{J}$

$\alpha \leftarrow \text{LineSearch}\big(\mathbf{x}^{(k)}, \Delta\mathbf{x}, \mathbf{J}\big)$

$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha\,\Delta\mathbf{x}$

$\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$

$\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$

end function

Incremental Potential Contact (IPC)

Incremental Potential Contact: Intersection- and Inversion-free, Large-Deformation Dynamics



https://ipc-sim.github.io

Incremental Potential Contact (IPC)

function IPCTimeStep($\mathbf{x}^-, \mathbf{v}^-$)

1: $\mathbf{x} \leftarrow \mathbf{x}^-$
2: **repeat**
3: $\mathbb{C} \leftarrow \text{ComputeCollisionSet}(\mathbf{x}, \hat{d})$
4: $B(\mathbf{x}) \leftarrow g(\mathbf{x}) + \kappa \sum_{c \in \mathbb{C}} b\big(d_c(\mathbf{x})\big)$
5: $\mathbf{J} \leftarrow \nabla B(\mathbf{x})$
6: $\bar{\mathbf{H}} \leftarrow \text{SPDProjection}\big(\nabla^2 B(\mathbf{x})\big)$
7: Solve $\bar{\mathbf{H}}\,\Delta\mathbf{x} = -\mathbf{J}$
8: $\alpha_{\max} \leftarrow \text{CCDStepSizeUpperBound}(\mathbf{x}, \Delta\mathbf{x})$
9: $\alpha \leftarrow \text{LineSearch}\big(\mathbf{x}, \Delta\mathbf{x}, \mathbf{J}, \alpha_{\max}\big)$
10: $\mathbf{x} \leftarrow \mathbf{x} + \alpha\,\Delta\mathbf{x}$
11: **until** $\frac{1}{h}\|\Delta\mathbf{x}\|_\infty < \epsilon$
12: $\mathbf{x}^+ \leftarrow \mathbf{x}$
13: $\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$

end function

Contacts as Energy

Surfaces that get close enough are added to the contact set. Each close pair adds a barrier energy that grows to infinity as the gap closes. Contact is just more energy in the same Newton solve, with no extra constraints.

Collision-Free Line Search

Collision detection along the whole step finds when the first collision would happen. The step is capped just before that moment. Every iterate stays free of intersections.

Convergence Check

...Stop when the Newton step, read as a velocity, becomes negligible.

SPD-Aware Newton's Method → Projective Dynamics (PD)

At each iteration *k* we have to:

- $\bar{\mathbf{H}}^{(k)} = \mathrm{SPDProjection}\left(\nabla^2 g(\mathbf{x}^{(k)})\right)$
- $\mathbf{C}^{(k)} = \mathrm{Factorize}\left(\bar{\mathbf{H}}^{(k)}\right)$
- $\Delta\mathbf{x} = \mathrm{SolveLinearSystem}\left(\mathbf{C}^{(k)}, -\mathbf{J}^{(k)}\right)$

These are very expensive operations (no Real-Time)

What is Factorization:

Factorization rewrites the matrix as a triangular matrix times its transpose:

$$\bar{\mathbf{H}} = \mathbf{C}\,\mathbf{C}^\top, \quad \mathbf{C} \text{ lower triangular}$$

The system is then solved with two cheap triangular substitutions:

$$\mathbf{C}\,\mathbf{y} = -\mathbf{J} \quad \longrightarrow \quad \mathbf{C}^\top \Delta\mathbf{x} = \mathbf{y}$$

The **factorization is the expensive part**.

Idea of PD: restrict the energies to a special form, so that the system matrix is **constant** and **SPD**, then **factorized once** and reused forever.

Projective Dynamics (PD)

Projective Dynamics: Fusing Constraint Projections for Fast Simulation

Sofien Bouaziz* EPFL
Sebastian Martin† VM Research
Tiantian Liu‡ University of Pennsylvania
Ladislav Kavan§ University of Pennsylvania
Mark Pauly¶ EPFL



[https://dl.acm.org/doi/10.1145/2601097.2601116]

Projective Dynamics (PD)

Each element energy is the squared distance between a linear function of the positions and a set of admissible configurations:

$$U_e(\mathbf{x}) = \min_{\mathbf{p}_e \in \mathcal{M}_e} \frac{k_e}{2} \left\| \mathbf{G}_e \, \mathbf{x} - \mathbf{p}_e \right\|^2$$

- $\mathbf{G}_e$: **constant** linear operator (selects and combines the element's particles)
- $\mathcal{M}_e$: **constraint manifold**, the set of undeformed (zero-energy) configurations
- $\mathbf{p}_e$: auxiliary variable, the **projection** of $\mathbf{G}_e \mathbf{x}$ onto $\mathcal{M}_e$

Projective Dynamics (PD)

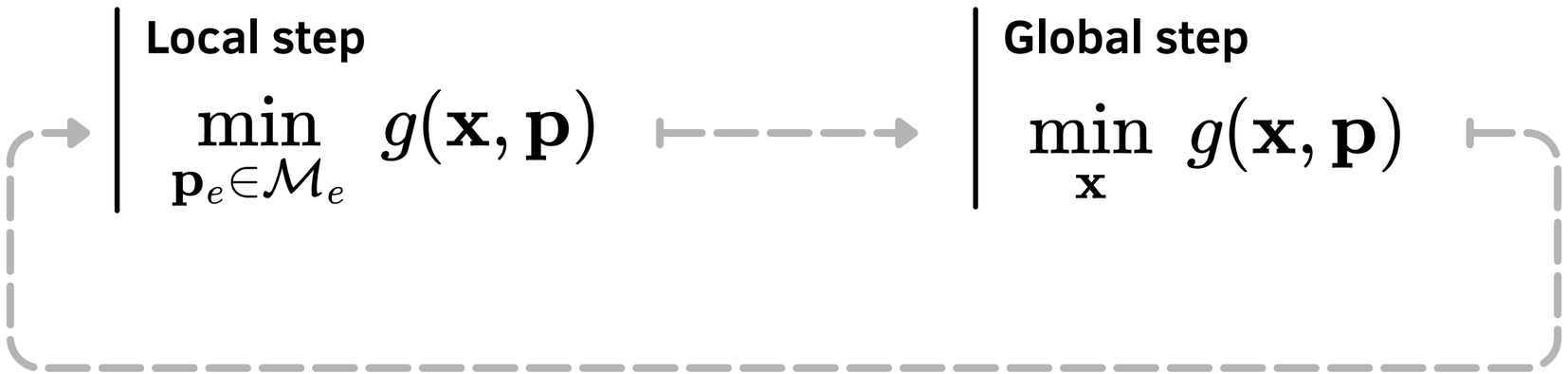
Augmented Energy:

$$g(\mathbf{x}, \mathbf{p}) = \frac{1}{2h^2}(\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + \sum_e \frac{k_e}{2} \left\| \mathbf{G}_e \, \mathbf{x} - \mathbf{p}_e \right\|^2$$

Minimization Problem:

$$\min_{\mathbf{x},\ \mathbf{p}_e \in \mathcal{M}_e} \quad g(\mathbf{x}, \mathbf{p})$$

We now minimize over two sets of variables, positions and projections. Instead of solving for both together, we alternate: minimize over one set while keeping the other fixed. This is alternating minimization, it turns the problem into a two-step iterative solver in which each step never increases the energy



Two-Step Iterative Solver

Projective Dynamics (PD)

$$\min_{\mathbf{x},\ \mathbf{p}_e \in \mathcal{M}_e} \ g(\mathbf{x}, \mathbf{p}) = \frac{1}{2h^2}(\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + \sum_e \frac{k_e}{2} \left\| \mathbf{G}_e\, \mathbf{x} - \mathbf{p}_e \right\|^2$$

Local step: fix $\mathbf{x}$, minimize over each $\mathbf{p}_e$. Each is a small **independent projection**, done in **parallel**:

$$\min_{\mathbf{p}_e \in \mathcal{M}_e} \ g(\mathbf{x}, \mathbf{p}) \quad \longrightarrow \quad \mathbf{p}_e = \mathrm{Proj}_{\mathcal{M}_e}\left(\mathbf{G}_e\, \mathbf{x}^{(k)}\right)$$

Global step: fix $\mathbf{p}_e$, minimize over $\mathbf{x}$. The objective is quadratic, so we set its gradient to zero, which gives the linear system:

$$\underbrace{\left[\frac{1}{h^2}\mathbf{M} + \sum_e k_e\, \mathbf{G}_e^\top \mathbf{G}_e\right]}_{\mathbf{L}_{\mathrm{PD}}\text{ : is constant and SPD}} \mathbf{x}^{(k+1)} = \underbrace{\frac{1}{h^2}\mathbf{M}\,\tilde{\mathbf{x}} + \sum_e k_e\, \mathbf{G}_e^\top \mathbf{p}_e}_{\mathbf{b}}$$

Projective Dynamics (PD)

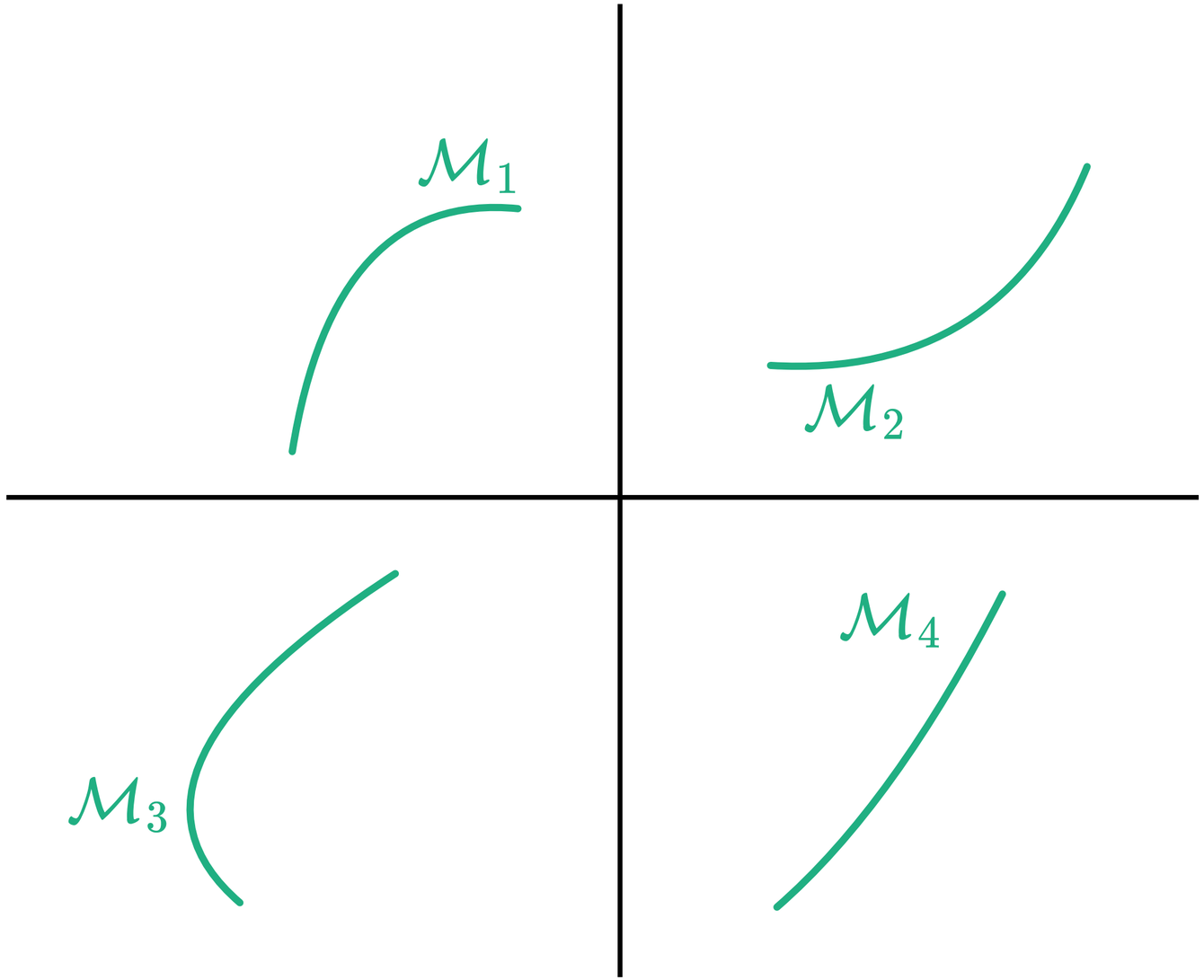
for $k \in [0, 1, \ldots, N-1]$ **do:**

  for each element e **do:**

    Compute $\mathbf{G}_e\,\mathbf{x}^{(k)}$

    $\mathbf{p}_e = \mathrm{Proj}_{\mathcal{M}_e}\left(\mathbf{G}_e\,\mathbf{x}^{(k)}\right)$

  Solve $\mathbf{L}_{\mathrm{PD}}\mathbf{x}^{(k+1)} = \mathbf{b}$



Projective Dynamics (PD)

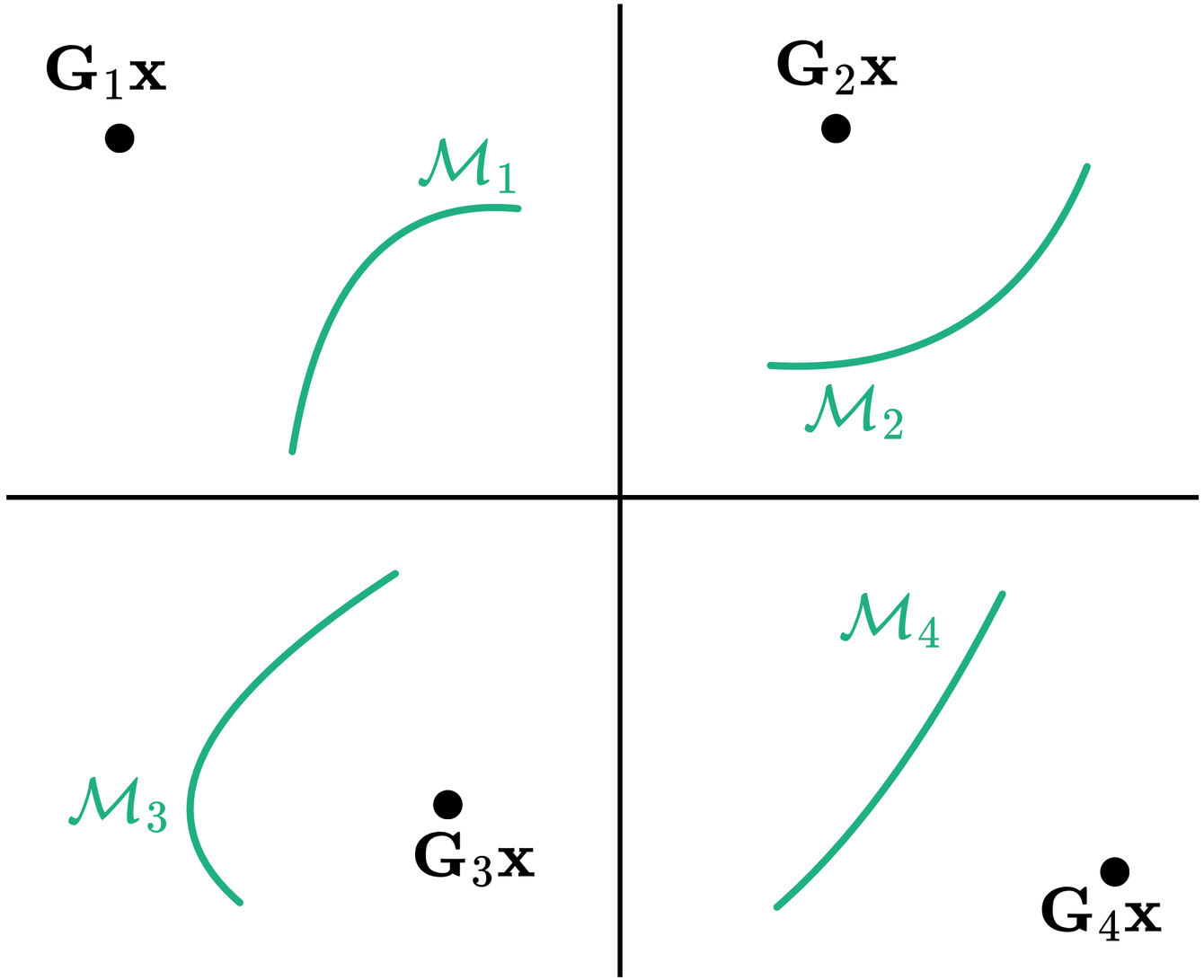
for $k \in [0, 1, \ldots, N-1]$ **do**:

  for each element e **do**:

    Compute $\mathbf{G}_e\,\mathbf{x}^{(k)}$

    $\mathbf{p}_e = \mathrm{Proj}_{\mathcal{M}_e}\left(\mathbf{G}_e\,\mathbf{x}^{(k)}\right)$

  Solve $\mathbf{L}_{\mathrm{PD}}\mathbf{x}^{(k+1)} = \mathbf{b}$



Projective Dynamics (PD)

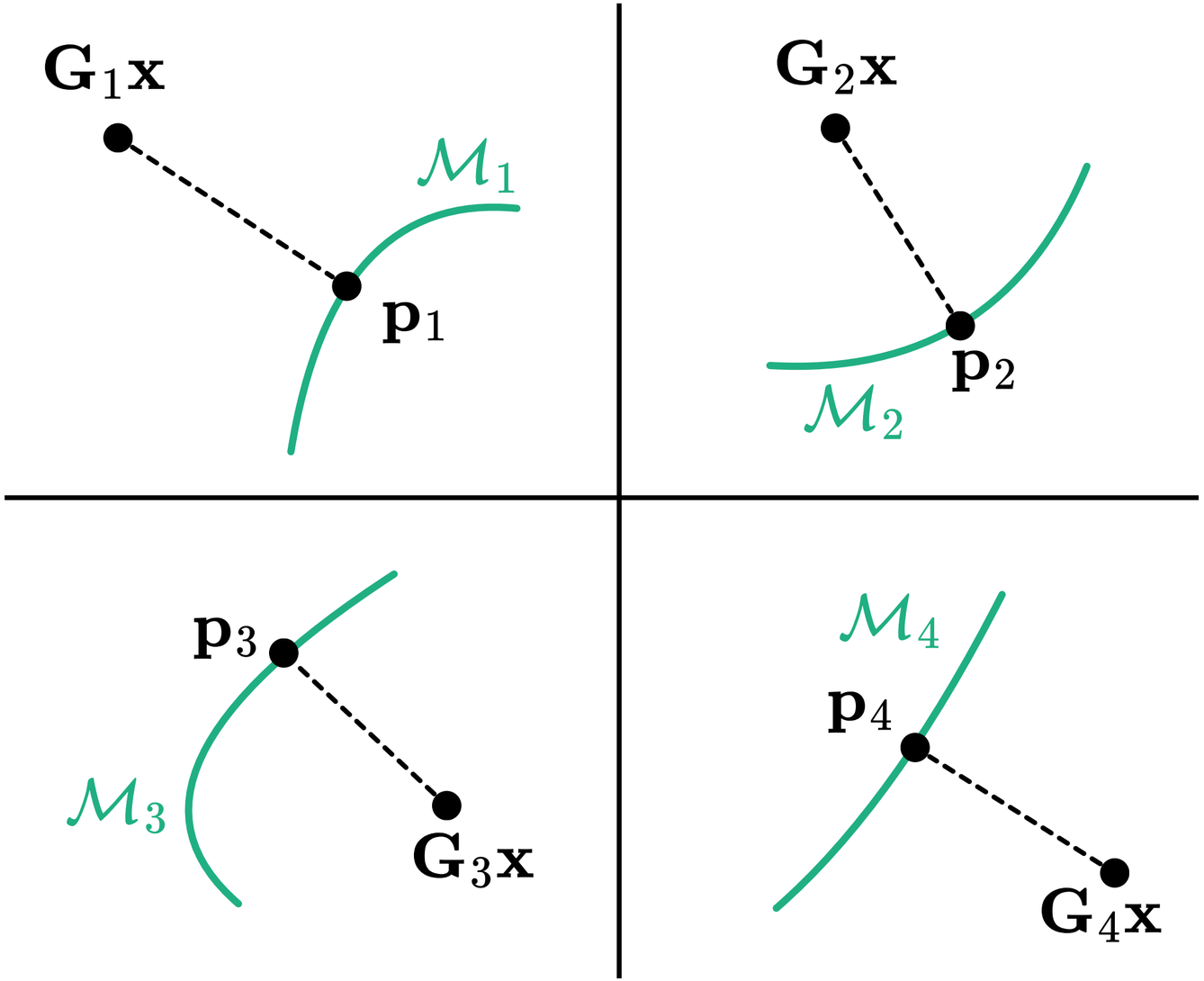
for $k \in [0, 1, \ldots, N-1]$ **do**:

  for each element e **do**:

    Compute $\mathbf{G}_e\,\mathbf{x}^{(k)}$

    $\mathbf{p}_e = \mathrm{Proj}_{\mathcal{M}_e}\left(\mathbf{G}_e\,\mathbf{x}^{(k)}\right)$

  Solve $\mathbf{L}_{\mathrm{PD}}\mathbf{x}^{(k+1)} = \mathbf{b}$



Projective Dynamics (PD)

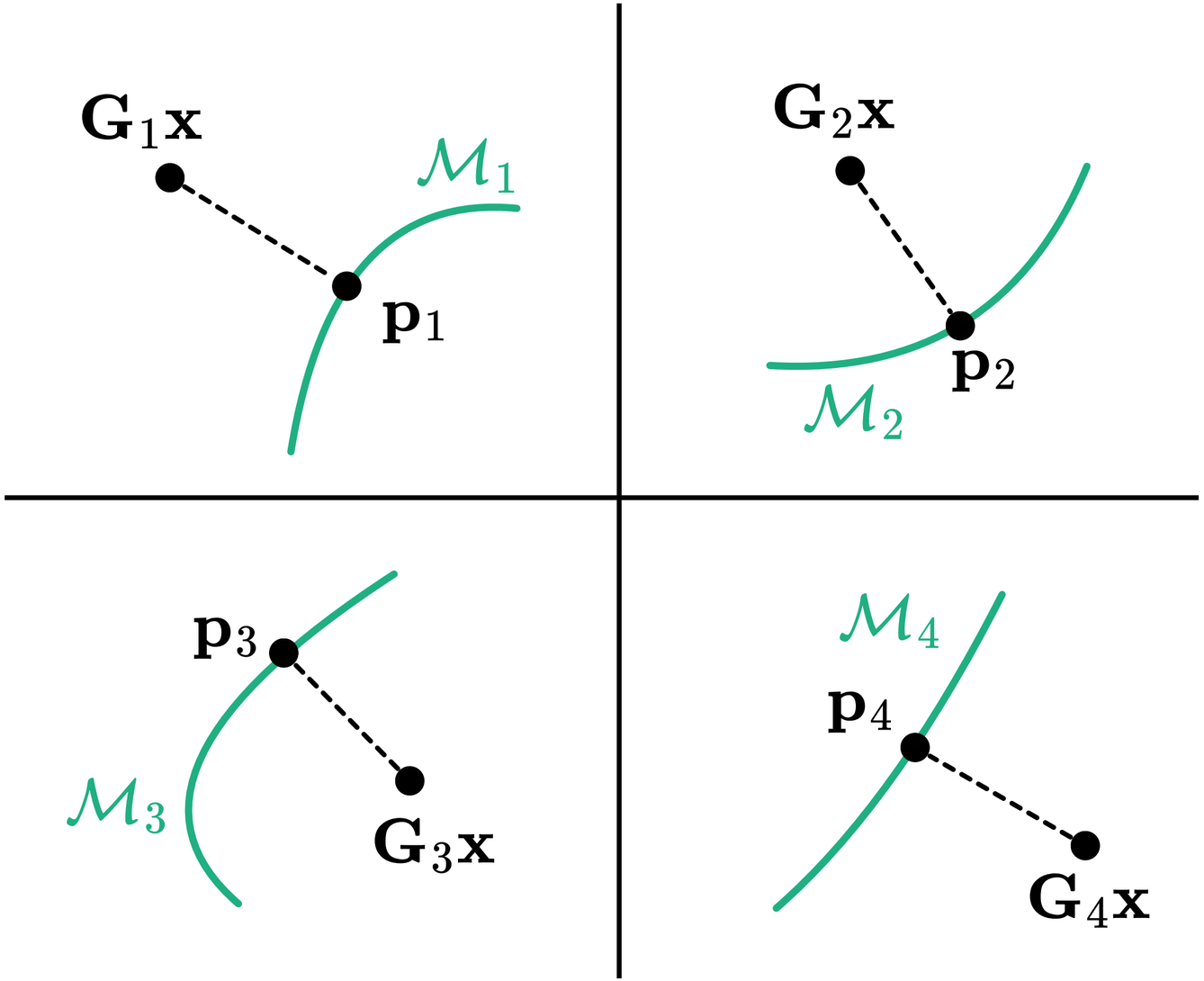
for $k \in [0, 1, \ldots, N-1]$ **do**:

  for each element e **do**:

    Compute $\mathbf{G}_e\,\mathbf{x}^{(k)}$

    $\mathbf{p}_e = \mathrm{Proj}_{\mathcal{M}_e}\left(\mathbf{G}_e\,\mathbf{x}^{(k)}\right)$

  Solve $\mathbf{L}_{\mathrm{PD}}\mathbf{x}^{(k+1)} = \mathbf{b}$



Projective Dynamics (PD)

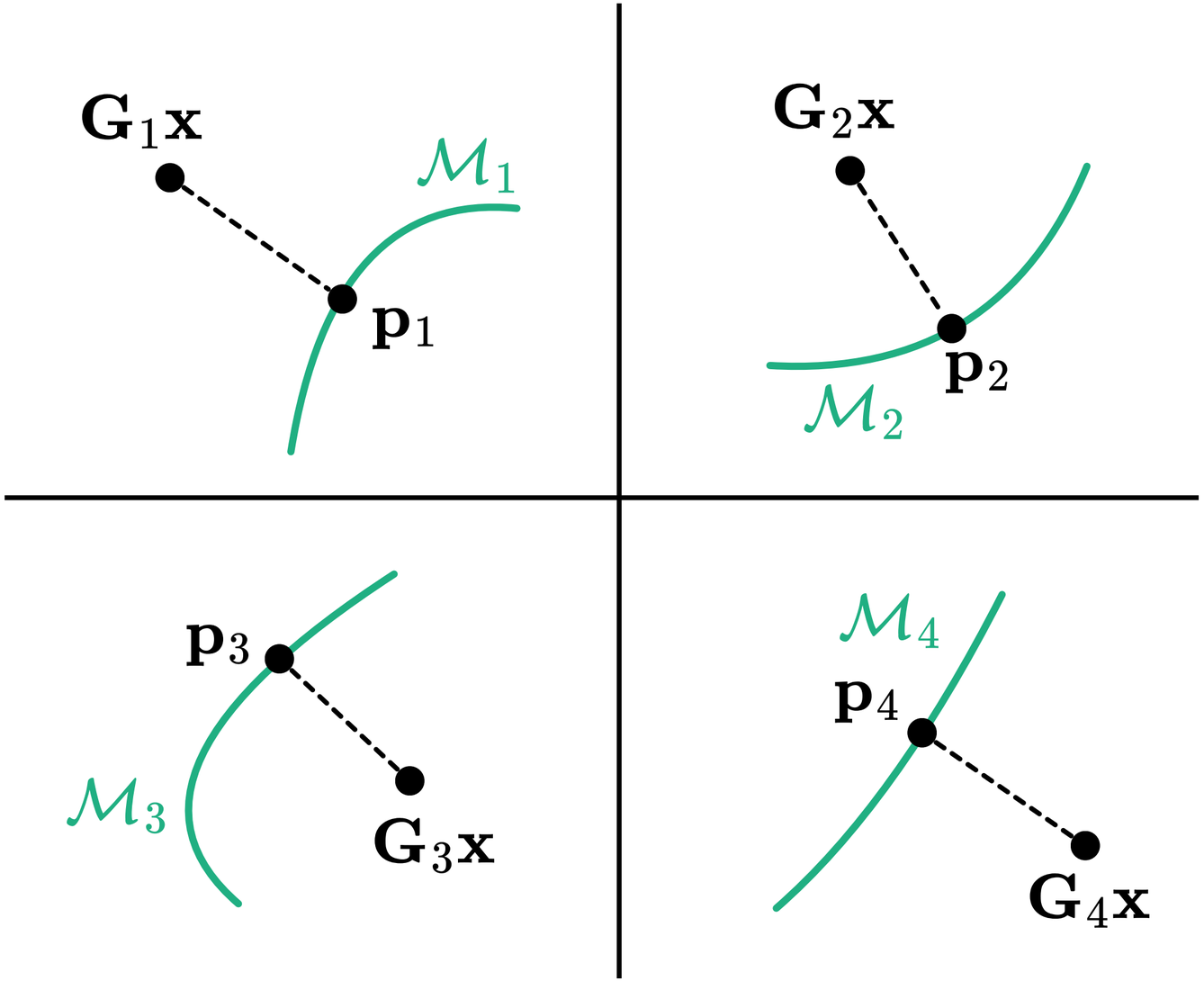
for $k \in [0, 1, \ldots, N-1]$ **do**:

  for each element e **do**:

    Compute $\mathbf{G}_e\,\mathbf{x}^{(k)}$

    $\mathbf{p}_e = \mathrm{Proj}_{\mathcal{M}_e}\left(\mathbf{G}_e\,\mathbf{x}^{(k)}\right)$

  Solve $\mathbf{L}_{\mathrm{PD}}\mathbf{x}^{(k+1)} = \mathbf{b}$



Projective Dynamics (PD)

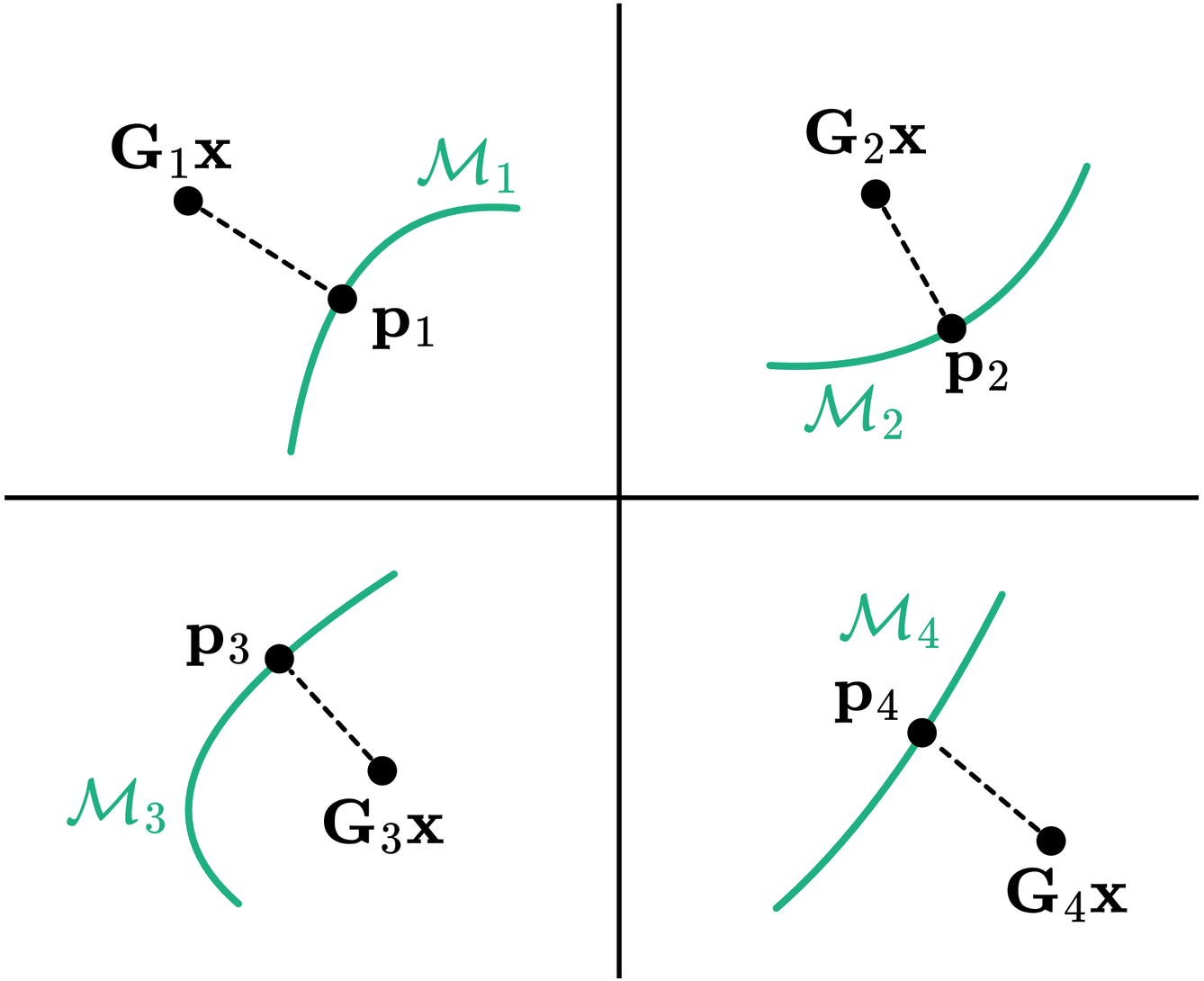
for $k \in [0, 1, \ldots, N-1]$ **do**:

  for each element e **do**:

    Compute $\mathbf{G}_e\,\mathbf{x}^{(k)}$

    $\mathbf{p}_e = \mathrm{Proj}_{\mathcal{M}_e}\left(\mathbf{G}_e\,\mathbf{x}^{(k)}\right)$

  Solve $\mathbf{L}_{\mathrm{PD}}\mathbf{x}^{(k+1)} = \mathbf{b}$



Projective Dynamics (PD)

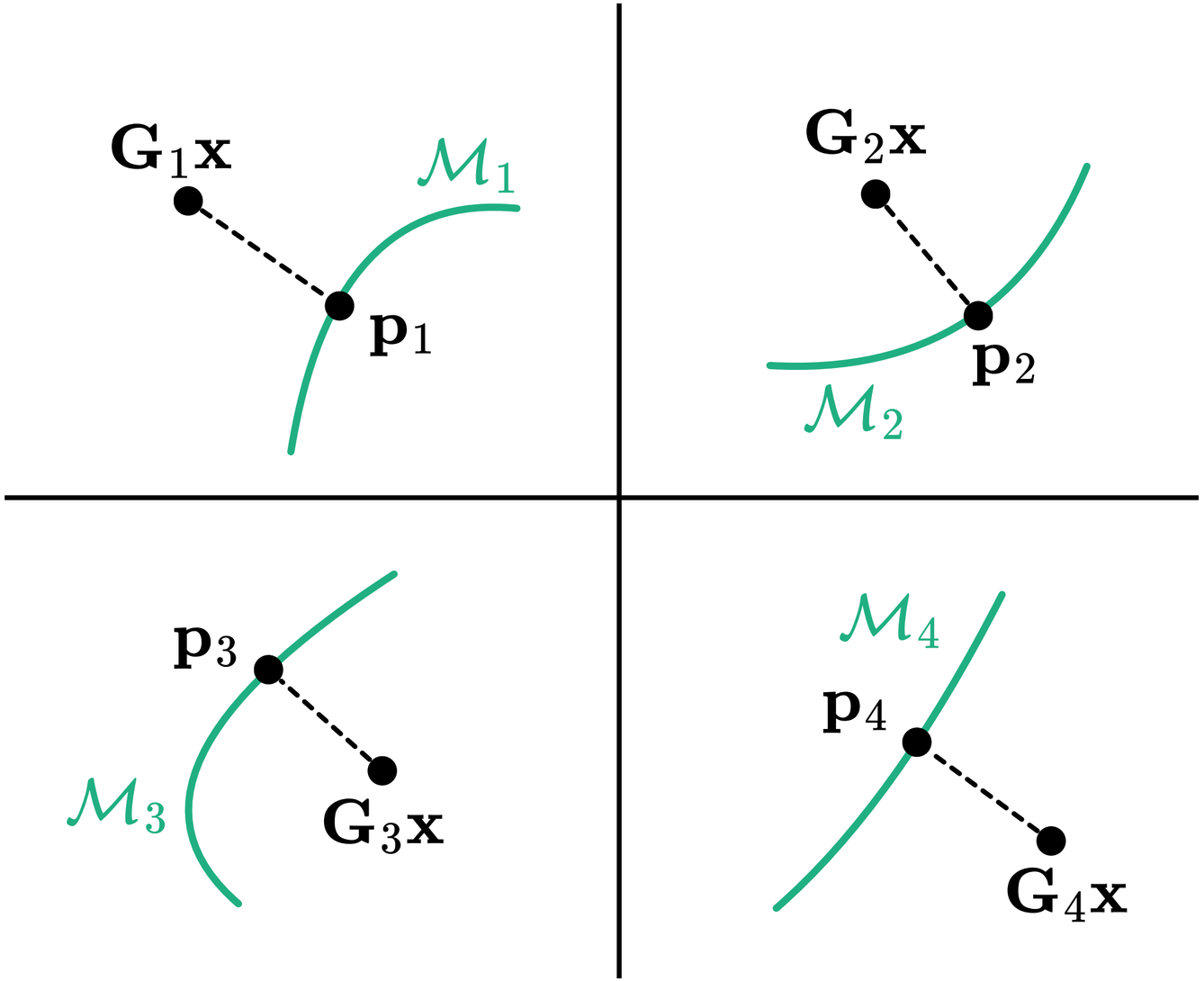
for $k \in [0, 1, \ldots, N-1]$ **do**:

  for each element e **do**:

    Compute $\mathbf{G}_e\,\mathbf{x}^{(k)}$

    $\mathbf{p}_e = \mathrm{Proj}_{\mathcal{M}_e}\left(\mathbf{G}_e\,\mathbf{x}^{(k)}\right)$

  Solve $\mathbf{L}_{\mathrm{PD}}\mathbf{x}^{(k+1)} = \mathbf{b}$



Projective Dynamics (PD)

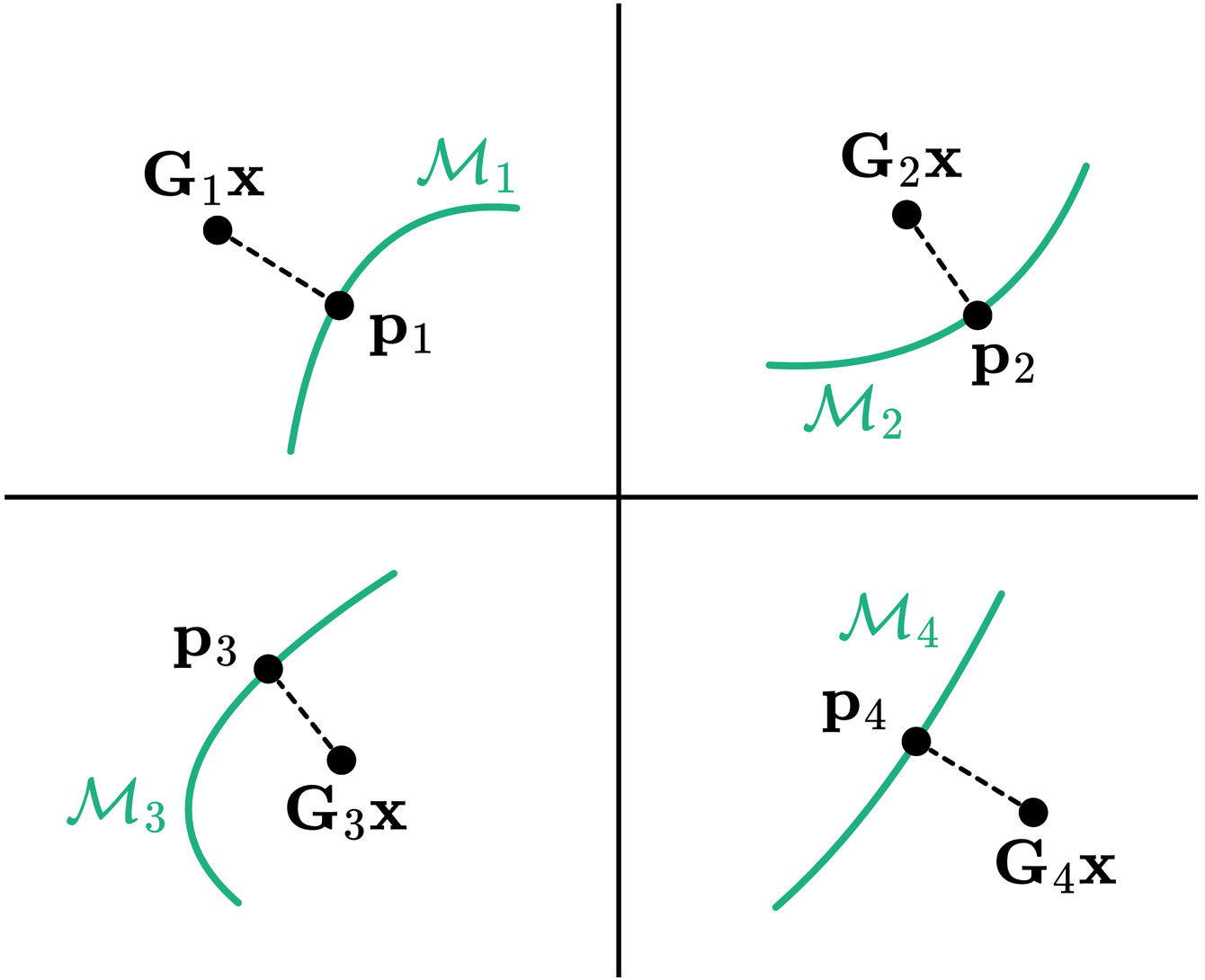
for $k \in [0, 1, \ldots, N-1]$ **do**:

  for each element e **do**:

    Compute $\mathbf{G}_e\,\mathbf{x}^{(k)}$

    $\mathbf{p}_e = \mathrm{Proj}_{\mathcal{M}_e}\left(\mathbf{G}_e\,\mathbf{x}^{(k)}\right)$

  Solve $\mathbf{L}_{\mathrm{PD}}\mathbf{x}^{(k+1)} = \mathbf{b}$



Projective Dynamics (PD)

function PROJECTIVEDYNAMICS($\tilde{\mathbf{x}}^-, \tilde{\mathbf{v}}^-$)

$\tilde{\mathbf{x}} \leftarrow \mathbf{x}^- + h\,\mathbf{v}^-$

Initialize $\mathbf{x}^{(0)}$

for $k = 0, 1, \ldots, N-1$ **do**

  for each element e **in parallel do**

    $\mathbf{p}_e \leftarrow \text{PROJ}_{\mathcal{M}_e}\left(\mathbf{G}_e\,\mathbf{x}^{(k)}\right)$

  $\mathbf{b} \leftarrow \frac{1}{h^2}\mathbf{M}\,\tilde{\mathbf{x}} + \sum_e k_e\,\mathbf{G}_e^\top \mathbf{p}_e$

  SOLVE $\mathbf{L}_{\text{PD}}\,\mathbf{x}^{(k+1)} = \mathbf{b}$

$\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$

$\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$

end function

Local step

Each element is projected onto its constraint set independently and in parallel.

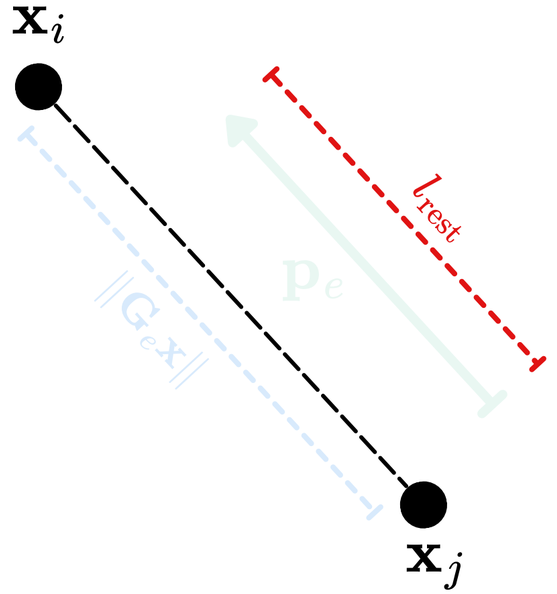
Global step

Here, the linear solve is fast because the constant system matrix $\mathbf{L}_{\text{PD}}$ has already been factorized once, at initialization.

Projective Dynamics (PD) Spring Constraint

Spring:

i, j : indices of particles l_{rest} : rest length k : stiffness



Projective Dynamics (PD) Spring Constraint

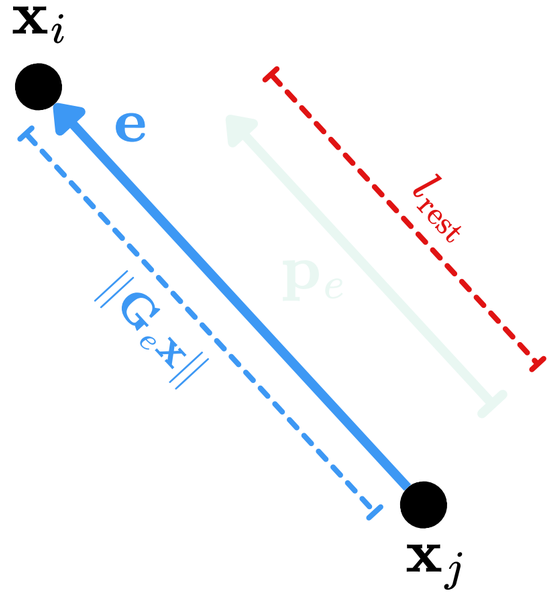
Spring:

i, j : indices of particles l_{rest} : rest length k : stiffness

Linear Operator:

$$\mathbf{G}_e \in \mathbb{R}^{3\times 3n}$$

$$\mathbf{G}_e\,\mathbf{x} = \begin{bmatrix} \cdots & \overset{i}{\mathbf{I}} & \cdots & \overset{j}{-\mathbf{I}} & \cdots \end{bmatrix}^\top \begin{bmatrix} \vdots \\ \mathbf{x}_i \\ \vdots \\ \mathbf{x}_j \\ \vdots \end{bmatrix} = \mathbf{x}_i - \mathbf{x}_j = \mathbf{e}$$



Projective Dynamics (PD) Spring Constraint

Spring:

i, j : indices of particles l_{rest} : rest length k : stiffness

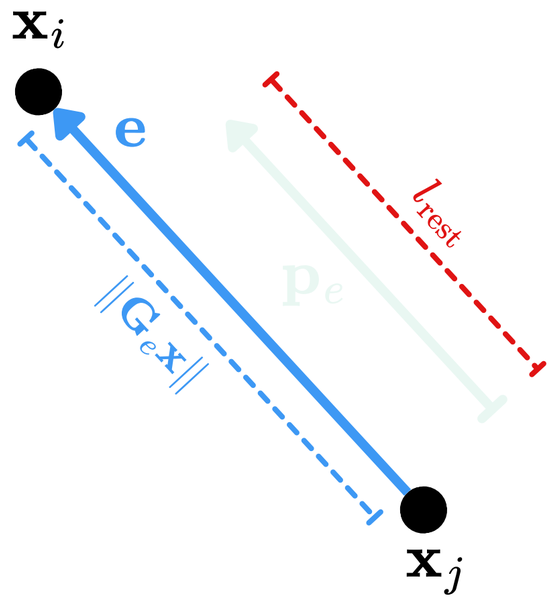
Linear Operator:

$$\mathbf{G}_e = \begin{bmatrix} \cdots & \overset{i}{\mathbf{I}} & \cdots & \overset{j}{-\mathbf{I}} & \cdots \end{bmatrix} \in \mathbb{R}^{3\times 3n}$$

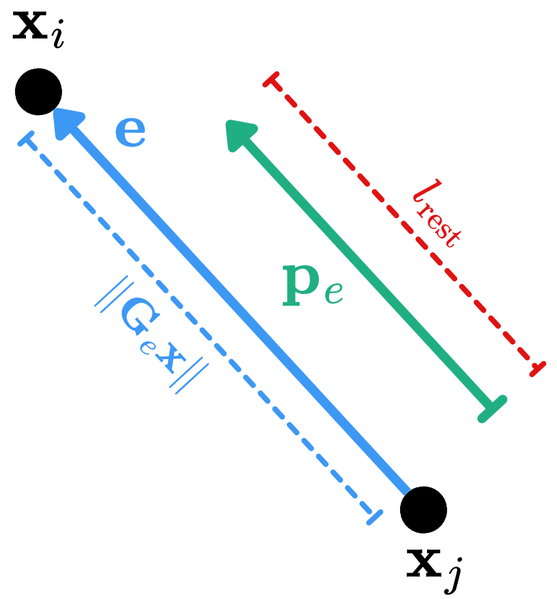
$$\mathbf{G}_e\mathbf{x} = \mathbf{x}_i - \mathbf{x}_j = \mathbf{e}$$

Constraint Manifold:

$$\mathcal{M}_e = \left\{ \mathbf{p} \in \mathbb{R}^3 \ : \ \|\mathbf{p}\| = l_{\text{rest}} \right\}$$



Projective Dynamics (PD) Spring Constraint



Spring:

i, j : indices of particles l_{rest} : rest length k : stiffness

Linear Operator:

$$\mathbf{G}_e = \begin{bmatrix} \cdots & \overset{i}{\mathbf{I}} & \cdots & \overset{j}{-\mathbf{I}} & \cdots \end{bmatrix} \in \mathbb{R}^{3\times 3n}$$

$$\mathbf{G}_e\mathbf{x} = \mathbf{x}_i - \mathbf{x}_j = \mathbf{e}$$

Constraint Manifold:

$$\mathcal{M}_e = \left\{ \mathbf{p} \in \mathbb{R}^3 \ : \ \|\mathbf{p}\| = l_{\text{rest}} \right\}$$

Local Step (Projection):

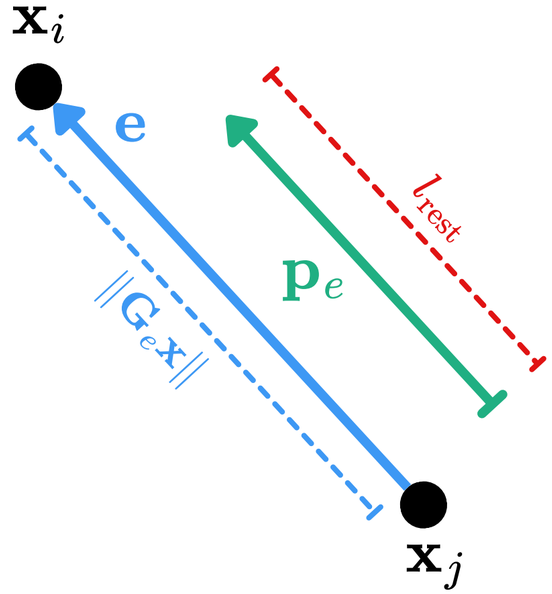
$$\mathbf{p}_e = l_{\text{rest}} \frac{\mathbf{G}_e\mathbf{x}}{\|\mathbf{G}_e\mathbf{x}\|}$$

Global Step:

$$\mathbf{L}_{\text{PD}} \mathrel{+}= k_e \, \mathbf{G}_e^\top \mathbf{G}_e$$

$$\mathbf{b} \mathrel{+}= k_e \, \mathbf{G}_e^\top \mathbf{p}_e$$

Projective Dynamics (PD) Spring Constraint



Spring:

i, j : indices of particles $\quad l_{\text{rest}}$: rest length $\quad k$: stiffness

Linear Operator:

$$\mathbf{G}_e = \begin{bmatrix} \cdots & \overset{i}{\mathbf{I}} & \cdots & \overset{j}{-\mathbf{I}} & \cdots \end{bmatrix} \in \mathbb{R}^{3\times 3n}$$

$$\mathbf{G}_e\mathbf{x} = \mathbf{x}_i - \mathbf{x}_j = \mathbf{e}$$

Constraint Manifold:

$$\mathcal{M}_e = \left\{ \mathbf{p} \in \mathbb{R}^3 \;:\; \|\mathbf{p}\| = l_{\text{rest}} \right\}$$

Local Step (Projection):

$$\mathbf{p}_e = l_{\text{rest}} \frac{\mathbf{G}_e\mathbf{x}}{\|\mathbf{G}_e\mathbf{x}\|}$$

Global Step:

$$\mathbf{L}_{\text{PD}} \mathrel{+}= k_e \mathbf{G}_e^\top \mathbf{G}_e \longrightarrow \begin{array}{c} \\ i \\ \\ j \\ \\ \end{array} \overset{\begin{array}{cccc} & i & & j \end{array}}{\begin{bmatrix} & \vdots & & \vdots & \\ \cdots & \mathbf{I} & \cdots & -\mathbf{I} & \cdots \\ & \vdots & & \vdots & \\ \cdots & -\mathbf{I} & \cdots & \mathbf{I} & \cdots \\ & \vdots & & \vdots & \end{bmatrix}} \in \mathbb{R}^{3n\times 3n}$$

$$\mathbf{b} \mathrel{+}= k_e \mathbf{G}_e^\top \mathbf{p}_e \longrightarrow \begin{bmatrix} \cdots & \mathbf{p}_e & \cdots & -\mathbf{p}_e & \cdots \end{bmatrix}^\top \in \mathbb{R}^{3n}$$

Differentiating Through Variational Implicit Euler Solver

Differentiating Through Variational Implicit Euler Solver

The loss is measured on the simulated trajectory. We know the gradient of the loss after a step and want it before the step:

$$\text{known: } \frac{\partial \phi}{\partial \mathbf{x}^+} \quad \longrightarrow \quad \text{wanted: } \frac{\partial \phi}{\partial \mathbf{x}^-}$$

Repeating this step by step, from the last to first frame, gives the gradient with respect to the initial state:

$$\frac{\partial \phi}{\partial \mathbf{x}_T} \xrightarrow{\text{step } T} \frac{\partial \phi}{\partial \mathbf{x}_{T-1}} \xrightarrow{\text{step } T-1} \frac{\partial \phi}{\partial \mathbf{x}_{T-2}} \longrightarrow \cdots \longrightarrow \frac{\partial \phi}{\partial \mathbf{x}_0}$$

The building block we would like to compute then is:

$$\frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-} \xrightarrow{\text{which through chain rule gives us:}} \frac{\partial \phi}{\partial \mathbf{x}^-} = \frac{\partial \phi}{\partial \mathbf{x}^+} \frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-}$$

Differentiating Through Variational Implicit Euler Solver

Energy and its Derivatives:

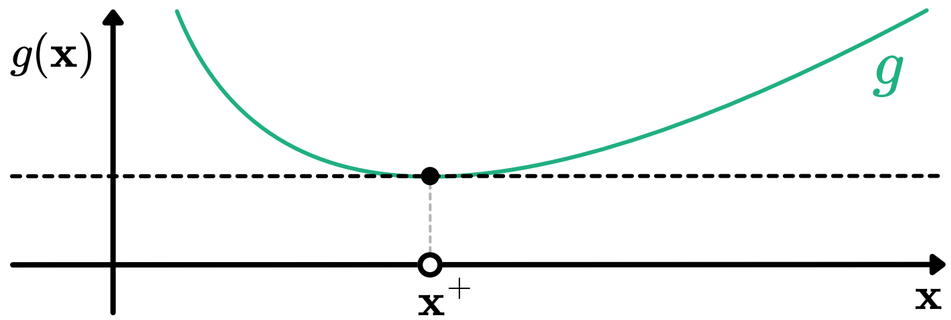
$$g(\mathbf{x}) = \frac{1}{2h^2}\left(\mathbf{x} - \tilde{\mathbf{x}}\right)^\top \mathbf{M}\left(\mathbf{x} - \tilde{\mathbf{x}}\right) + U(\mathbf{x})$$

$$\mathbf{J} = \nabla g(\mathbf{x}) = \frac{1}{h^2}\mathbf{M}\left(\mathbf{x} - \tilde{\mathbf{x}}\right) + \nabla U(\mathbf{x})$$

$$\mathbf{H} = \nabla^2 g(\mathbf{x}) = \frac{1}{h^2}\mathbf{M} + \nabla^2 U(\mathbf{x})$$

At the end of the step the converged solution $\mathbf{x}^+$ satisfy the **optimality condition**:

$$\nabla g(\mathbf{x}^+) = \mathbf{0}$$



This holds at convergence whatever solver was used (Newton, PD, ...)
We differentiate this condition (**implicit differentiation**), not the solver's iterations.

Differentiating Through Variational Implicit Euler Solver

Energy and its Derivatives:

$$g(\mathbf{x}) = \frac{1}{2h^2}\left(\mathbf{x} - \tilde{\mathbf{x}}\right)^\top \mathbf{M}\left(\mathbf{x} - \tilde{\mathbf{x}}\right) + U(\mathbf{x})$$

$$\mathbf{J} = \nabla g(\mathbf{x}) = \frac{1}{h^2}\mathbf{M}\left(\mathbf{x} - \tilde{\mathbf{x}}\right) + \nabla U(\mathbf{x})$$

$$\mathbf{H} = \nabla^2 g(\mathbf{x}) = \frac{1}{h^2}\mathbf{M} + \nabla^2 U(\mathbf{x})$$

Differentiating the optimality condition with respect to $\mathbf{x}^-$ gives:

$$\underbrace{\left[\frac{1}{h^2}\mathbf{M} + \nabla^2 U(\mathbf{x}^+)\right]}_{\mathbf{H} = \nabla^2 g(\mathbf{x}^+)} \frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-} - \frac{1}{h^2}\mathbf{M} = \mathbf{0} \quad \Longrightarrow \quad \frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-} = \frac{1}{h^2}\,\mathbf{H}^{-1}\mathbf{M}$$

Then the chain rule gives:

$$\frac{\partial \phi}{\partial \mathbf{x}^-} = \frac{\partial \phi}{\partial \mathbf{x}^+}\,\frac{\partial \mathbf{x}^+}{\partial \mathbf{x}^-} = \frac{1}{h^2}\,\frac{\partial \phi}{\partial \mathbf{x}^+}\,\mathbf{H}^{-1}\,\mathbf{M}$$

computed once at convergence

Differentiating Through Variational Implicit Euler Solver

$$\frac{\partial \phi}{\partial \mathbf{x}^-} = \frac{1}{h^2} \underbrace{\frac{\partial \phi}{\partial \mathbf{x}^+} \mathbf{H}^{-1}}_{\mathbf{z}^\top} \mathbf{M}$$

We never compute the dense matrix $\mathbf{H}^{-1}$. Instead, we group the first two factors into the **adjoint vector** $\mathbf{z}$, which comes from a single linear solve:

$$\mathbf{z}^\top = \frac{\partial \phi}{\partial \mathbf{x}^+} \mathbf{H}^{-1} \quad \Longleftrightarrow \quad \mathbf{H}\,\mathbf{z} = \left(\frac{\partial \phi}{\partial \mathbf{x}^+} \right)^\top \longmapsto$$

Single Linear System Solve

Then both positions and velocities gradients are cheap matrix–vector products:

$$\frac{\partial \phi}{\partial \mathbf{x}^-} = \frac{1}{h^2} \mathbf{z}^\top \mathbf{M} \qquad \frac{\partial \phi}{\partial \mathbf{v}^-} = \frac{1}{h} \mathbf{z}^\top \mathbf{M}$$

Differentiating Through Variational Implicit Euler Solver

function NEWTONSTEP($\mathbf{x}^-, \mathbf{v}^-$)

1: Initialize $\mathbf{x}$
2: **for** $k = 0, 1, \ldots, N-1$ **do**
3: BUILD $\mathbf{J}, \mathbf{H}$
4: SOLVE $\mathbf{H}\,\Delta\mathbf{x} = -\mathbf{J}$
5: $\mathbf{x} \leftarrow \mathbf{x} + \Delta\mathbf{x}$
6: $\mathbf{x}^+ \leftarrow \mathbf{x}$
7: $\mathbf{v}^+ \leftarrow \frac{1}{h}(\mathbf{x}^+ - \mathbf{x}^-)$

end function

Forward:

N iterations, and each one builds and solves with a new Hessian.

function BACKWARDSTEP($\mathbf{x}^+, \frac{\partial \phi}{\partial \mathbf{x}^+}$)

1: BUILD $\mathbf{H} = \nabla^2 g(\mathbf{x}^+)$
2: SOLVE $\mathbf{H}\,\mathbf{z} = \left(\frac{\partial \phi}{\partial \mathbf{x}^+}\right)^\top$
3: $\frac{\partial \phi}{\partial \mathbf{x}^-} \leftarrow \frac{1}{h^2}\,\mathbf{z}^\top \mathbf{M}$
4: $\frac{\partial \phi}{\partial \mathbf{v}^-} \leftarrow \frac{1}{h}\,\mathbf{z}^\top \mathbf{M}$

end function

Backward:

One solve with the same Hessian built at convergence, costs about one Newton iteration.

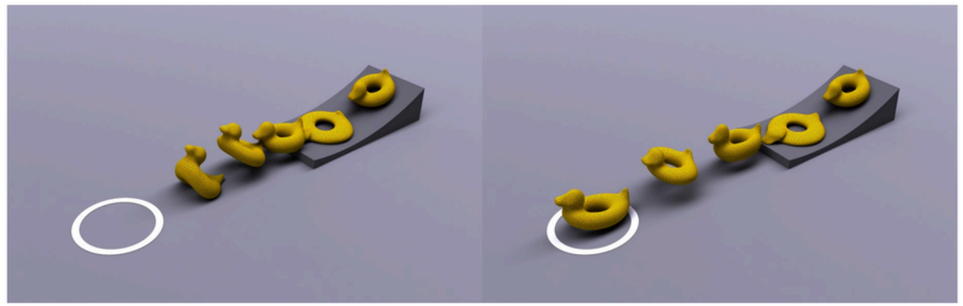
The Question then is:

Since in PD we never construct the true Hessian $\mathbf{H}$ explicitly, how do we compute the adjoint vector?

Differentiable Projective Dynamics: DiffPD

DiffPD: Differentiable Projective Dynamics

TAO DU, KUI WU, PINGCHUAN MA, SEBASTIEN WAH, and ANDREW SPIELBERG, MIT CSAIL
DANIELA RUS and WOJCIECH MATUSIK, MIT CSAIL



[https://diffpd.csail.mit.edu]

Differentiable Projective Dynamics: DiffPD

We should find a way to compute: $\mathbf{H}_{\text{PD}} = \nabla^2 g_{\text{PD}}(\mathbf{x})$

We need it for: $\mathbf{H}\,\mathbf{z} = \left(\frac{\partial \phi}{\partial \mathbf{x}^+}\right)^\top$

PD energy is: $g(\mathbf{x}, \mathbf{p}) = \frac{1}{2h^2}(\mathbf{x} - \tilde{\mathbf{x}})^\top \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + \underbrace{\sum \frac{k_e}{2} \left\| \mathbf{G}_e\,\mathbf{x} - \mathbf{p}_e \right\|^2}_{U}$

But the projections really depends on the positions:

$$\mathbf{p}_e(\mathbf{x}) = \text{Proj}_{\mathcal{M}_e}(\mathbf{G}_e \mathbf{x}) \longmapsto U(\mathbf{x}) = \sum \frac{k_e}{2} \left\| \mathbf{G}_e\,\mathbf{x} - \mathbf{p}_e(\mathbf{x}) \right\|^2$$

From which we can derive the first and second derivatives:

$$\nabla U(\mathbf{x}) = \sum k_e\, \mathbf{G}_e^\top \left(\mathbf{G}_e\,\mathbf{x} - \mathbf{p}_e \right) \qquad \nabla^2 U(\mathbf{x}) = \sum k_e\, \mathbf{G}_e^\top \mathbf{G}_e \; - \; \sum k_e\, \mathbf{G}_e^\top \frac{\partial\, \mathbf{p}_e}{\partial\, \mathbf{x}}$$

And then we can compute the full real PD Hessian:

$$\mathbf{H}_{\text{PD}} = \frac{1}{h^2}\mathbf{M} + \sum k_e\, \mathbf{G}_e^\top \mathbf{G}_e - \sum k_e\, \mathbf{G}_e^\top \frac{\partial\, \mathbf{p}_e}{\partial\, \mathbf{x}}$$

Differentiable Projective Dynamics: DiffPD

$$\mathbf{H}_{\mathrm{PD}} = \underbrace{\frac{1}{h^2}\mathbf{M} + \sum k_e\, \mathbf{G}_e^\top \mathbf{G}_e}_{\mathbf{L}_{\mathrm{PD}}} - \underbrace{\sum k_e\, \mathbf{G}_e^\top \frac{\partial \mathbf{p}_e}{\partial \mathbf{x}}}_{\Delta\mathbf{L}} = \mathbf{L}_{\mathrm{PD}} - \Delta\mathbf{L}$$

- $\mathbf{L}_{\mathrm{PD}}$: constant, and already factorized in the forward simulation.
- $\Delta\mathbf{L}$: contains all the nonlinearity; it is a sum of small per-element terms, computed in parallel.

$$\frac{\partial \mathbf{p}_e}{\partial \mathbf{x}} = \underbrace{\frac{\partial\, \mathrm{Proj}_{\mathcal{M}_e}}{\partial (\mathbf{G}_e \mathbf{x})}}_{\boldsymbol{\Gamma}_e} \mathbf{G}_e = \boldsymbol{\Gamma}_e\, \mathbf{G}_e \quad \Rightarrow \quad \Delta\mathbf{L} = \sum_e k_e\, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e\, \mathbf{G}_e$$

Differentiable Projective Dynamics: DiffPD

Substituting the split into the adjoint system $\mathbf{H}\,\mathbf{z} = \left(\frac{\partial\,\phi}{\partial\,\mathbf{x}^{+}}\right)^{\top}$ we obtain:

$$\left(\mathbf{L}_{\mathrm{PD}} - \Delta\mathbf{L}\right)\mathbf{z} = \left(\frac{\partial\,\phi}{\partial\,\mathbf{x}^{+}}\right)^{\top}$$

Moving $\Delta\mathbf{L}\,\mathbf{z}$ to the right-hand side and evaluating it at the previous iterate gives a **fixed-point iteration**:

$$\mathbf{L}_{\mathrm{PD}}\,\mathbf{z}^{(k+1)} = \Delta\mathbf{L}\,\mathbf{z}^{(k)} + \left(\frac{\partial\,\phi}{\partial\,\mathbf{x}^{+}}\right)^{\top}$$

This has the same local/global structure as forward PD:

Local step:

$$\Delta\mathbf{L}\,\mathbf{z}^{(k)} = \sum k_e\,\mathbf{G}_e^{\top}\,\frac{\partial\,\mathbf{p}_e}{\partial\,\mathbf{x}}\,\mathbf{z}^{(k)}$$

Computed element by element and in parallel, using only the small projection Jacobians.

Global step:

$$\text{Solve}\quad \mathbf{L}_{\mathrm{PD}}\,\mathbf{z}^{(k+1)} = \dots$$

One back-substitution with the already factorized $\mathbf{L}_{\mathrm{PD}}$ in the forward pass; no new matrix is assembled or factorized.

Differentiable Projective Dynamics: DiffPD

Backward Algorithm

function DIFFPDBACKWARDSTEP($\mathbf{x}^+, \frac{\partial \phi}{\partial \mathbf{x}^+}$)

1: **for** each element e **in parallel do**

2: $\quad \boldsymbol{\Gamma}_e \leftarrow \frac{\partial \text{PROJ}_{\mathcal{M}_e}}{\partial (\mathbf{G}_e \mathbf{x})}$

3: Initialize $\mathbf{z}$

4: **for** $k = 0, 1, \ldots, K-1$ **do**

5: $\quad \mathbf{b} \leftarrow \sum_e k_e \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \mathbf{G}_e \mathbf{z}$

6: $\quad$ SOLVE $\mathbf{L}_{\text{PD}} \mathbf{z} = \mathbf{b} + \left(\frac{\partial \phi}{\partial \mathbf{x}^+}\right)^\top$

7: $\frac{\partial \phi}{\partial \mathbf{x}^-} \leftarrow \frac{1}{h^2} \mathbf{z}^\top \mathbf{M}$

8: $\frac{\partial \phi}{\partial \mathbf{v}^-} \leftarrow \frac{1}{h} \mathbf{z}^\top \mathbf{M}$

end function

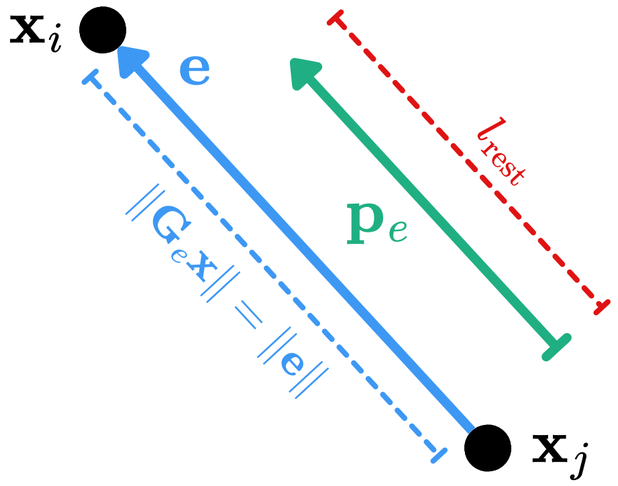
Compute Projection derivatives **at convergence** that we constantly reuse during the Local-Global steps.

Local step

Global step

Differentiable Projective Dynamics: DiffPD

Spring Constraint



Spring:

i, j : indices of particles $\quad l_{\text{rest}}$: rest length $\quad k$: stiffness

$$\mathbf{G}_e\mathbf{x} = \mathbf{x}_i - \mathbf{x}_j = \mathbf{e}$$

Projection:

$$\mathbf{p}_e = \text{Proj}_{\mathcal{M}_e}(\mathbf{e}) = l_{\text{rest}}\,\hat{\mathbf{e}}, \quad \hat{\mathbf{e}} = \frac{\mathbf{e}}{\|\mathbf{e}\|}$$

Projection Derivative:

$$\frac{\partial \hat{\mathbf{e}}}{\partial \mathbf{e}} = \frac{1}{\|\mathbf{e}\|}\left(\mathbf{I} - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top\right)$$

$$\mathbf{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial \mathbf{e}} = \frac{l_{\text{rest}}}{\|\mathbf{e}\|}\left(\mathbf{I} - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top\right) \in \mathbb{R}^{3\times 3}$$

Hessian Correction:

$$\Delta\mathbf{L}_e = k_e\,\mathbf{G}_e^\top\mathbf{\Gamma}_e\,\mathbf{G}_e = \begin{array}{c} \\ \\ \cdots \\ \\ \cdots \\ \\ \end{array}\!\!\begin{array}{c} \begin{array}{cccc} \quad i & & \quad j & \end{array} \\ \begin{bmatrix} \vdots & & \vdots & \\ \mathbf{\Gamma}_e & \cdots & -\mathbf{\Gamma}_e & \cdots \\ \vdots & & \vdots & \\ -\mathbf{\Gamma}_e & \cdots & \mathbf{\Gamma}_e & \cdots \\ \vdots & & \vdots & \end{bmatrix} \end{array}\!\!\begin{array}{l} \\ \\ i \\ \\ j \\ \\ \end{array}$$

$$\in \mathbb{R}^{3n\times 3n}$$

Differentiable Projective Dynamics: DiffPD

Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.
At convergence it holds that:

$$\mathbf{L}_{\mathrm{PD}}(\theta)\,\mathbf{x}^{+} = \mathbf{b}\left(\mathbf{x}^{+}, \theta\right)$$

Differentiable Projective Dynamics: DiffPD

Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.
At convergence it holds that:

$$\mathbf{L}_{\mathrm{PD}}(\theta)\,\mathbf{x}^{+} = \mathbf{b}\left(\mathbf{x}^{+}, \theta\right)$$

Then we:

- **Differentiate** the converged equation with respect to θ.

$$\frac{\partial \mathbf{L}_{\mathrm{PD}}}{\partial \theta}\,\mathbf{x}^{+} + \mathbf{L}_{\mathrm{PD}}\,\frac{d\,\mathbf{x}^{+}}{d\,\theta} = \frac{\partial \mathbf{b}}{\partial\,\mathbf{x}^{+}}\,\frac{d\,\mathbf{x}^{+}}{d\,\theta} + \frac{\partial \mathbf{b}}{\partial\,\theta}$$

Differentiable Projective Dynamics: DiffPD

Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.
At convergence it holds that:

$$\mathbf{L}_{\mathrm{PD}}(\theta)\,\mathbf{x}^{+} = \mathbf{b}\left(\mathbf{x}^{+}, \theta\right)$$

Then we:

- **Differentiate** the converged equation with respect to θ.
- **Collect** the unknown sensitivity.

$$\underbrace{\left(\mathbf{L}_{\mathrm{PD}} - \frac{\partial \mathbf{b}}{\partial \mathbf{x}^{+}}\right)}_{\mathbf{L}_{\mathrm{PD}} - \Delta\mathbf{L} \,=\, \mathbf{H}_{\mathrm{PD}}} \frac{d\,\mathbf{x}^{+}}{d\,\theta} = \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\mathrm{PD}}}{\partial \theta}\,\mathbf{x}^{+}$$

Differentiable Projective Dynamics: DiffPD

Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.
At convergence it holds that:

$$\mathbf{L}_{\mathrm{PD}}(\theta)\,\mathbf{x}^{+} = \mathbf{b}\left(\mathbf{x}^{+}, \theta\right)$$

Then we:

- **Differentiate** the converged equation with respect to θ.
- **Collect** the unknown sensitivity.

$$\underbrace{\left(\mathbf{L}_{\mathrm{PD}} - \frac{\partial \mathbf{b}}{\partial \mathbf{x}^{+}}\right)}_{\mathbf{L}_{\mathrm{PD}} - \Delta\mathbf{L} \,=\, \mathbf{H}_{\mathrm{PD}}} \frac{d\,\mathbf{x}^{+}}{d\,\theta} = \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\mathrm{PD}}}{\partial \theta}\,\mathbf{x}^{+}$$

- **Recognize** the same adjoint vector we already computed in the chain rule:

$$\frac{d\,\phi}{d\,\theta} = \frac{\partial\,\phi}{\partial\,\mathbf{x}^{+}}\,\frac{d\,\mathbf{x}^{+}}{d\,\theta}$$

Differentiable Projective Dynamics: DiffPD

Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.
At convergence it holds that:

$$\mathbf{L}_{\mathrm{PD}}(\theta)\,\mathbf{x}^{+} = \mathbf{b}\left(\mathbf{x}^{+}, \theta\right)$$

Then we:

- **Differentiate** the converged equation with respect to θ.
- **Collect** the unknown sensitivity.

$$\underbrace{\left(\mathbf{L}_{\mathrm{PD}} - \frac{\partial \mathbf{b}}{\partial \mathbf{x}^{+}}\right)}_{\mathbf{L}_{\mathrm{PD}} - \Delta\mathbf{L} \,=\, \mathbf{H}_{\mathrm{PD}}} \frac{d\,\mathbf{x}^{+}}{d\,\theta} = \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\mathrm{PD}}}{\partial \theta}\,\mathbf{x}^{+}$$

- **Recognize** the same adjoint vector we already computed in the chain rule:

$$\frac{d\,\phi}{d\,\theta} = \frac{\partial \phi}{\partial \mathbf{x}^{+}}\,\frac{d\,\mathbf{x}^{+}}{d\,\theta} = \underbrace{\frac{\partial \phi}{\partial \mathbf{x}^{+}}\,\mathbf{H}_{\mathrm{PD}}^{-1}}_{\mathbf{z}^{\top}} \left(\frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\mathrm{PD}}}{\partial \theta}\,\mathbf{x}^{+}\right)$$

Differentiable Projective Dynamics: DiffPD

Gradient with Respect to a General Parameter

A parameter (stiffness, rest length, mass, ...) can enter both the matrix and the right-hand side.
At convergence it holds that:

$$\mathbf{L}_{\mathrm{PD}}(\theta)\,\mathbf{x}^{+} = \mathbf{b}\big(\mathbf{x}^{+}, \theta\big)$$

Then we:

- **Differentiate** the converged equation with respect to θ.
- **Collect** the unknown sensitivity.

$$\underbrace{\left(\mathbf{L}_{\mathrm{PD}} - \frac{\partial \mathbf{b}}{\partial \mathbf{x}^{+}}\right)}_{\mathbf{L}_{\mathrm{PD}} - \Delta\mathbf{L} \,=\, \mathbf{H}_{\mathrm{PD}}} \frac{d\,\mathbf{x}^{+}}{d\,\theta} = \frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\mathrm{PD}}}{\partial \theta}\,\mathbf{x}^{+}$$

- **Recognize** the same adjoint vector we already computed in the chain rule:

$$\frac{d\,\phi}{d\,\theta} = \frac{\partial \phi}{\partial \mathbf{x}^{+}}\,\frac{d\,\mathbf{x}^{+}}{d\,\theta} = \underbrace{\frac{\partial \phi}{\partial \mathbf{x}^{+}}\,\mathbf{H}_{\mathrm{PD}}^{-1}}_{\mathbf{z}^{\top}} \left(\frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\mathrm{PD}}}{\partial \theta}\,\mathbf{x}^{+}\right) = \mathbf{z}^{\top}\left(\frac{\partial \mathbf{b}}{\partial \theta} - \frac{\partial \mathbf{L}_{\mathrm{PD}}}{\partial \theta}\,\mathbf{x}^{+}\right)$$

Differentiable Projective Dynamics: DiffPD

Gradient with Respect to a General Parameter

$$\mathbf{L}_{\mathrm{PD}} = \tfrac{1}{h^2}\mathbf{M} + \textstyle\sum_e k_e\, \mathbf{G}_e^\top \mathbf{G}_e$$

$$\mathbf{b} = \tfrac{1}{h^2}\mathbf{M}\,\tilde{\mathbf{x}} + \textstyle\sum_e k_e\, \mathbf{G}_e^\top \mathbf{p}_e$$

$$\frac{d\,\phi}{d\,\theta} = \mathbf{z}^\top \left(\frac{\partial\,\mathbf{b}}{\partial\,\theta} - \frac{\partial\,\mathbf{L}_{\mathrm{PD}}}{\partial\,\theta}\,\mathbf{x}^+ \right)$$

θ	$\dfrac{\partial\,\mathbf{b}}{\partial\,\theta}$	$\dfrac{\partial\,\mathbf{L}_{\mathrm{PD}}}{\partial\,\theta}$	$\dfrac{d\,\phi}{d\,\theta}$
$\mathbf{x}^-$	$\frac{1}{h^2}\mathbf{M}$	$\mathbf{0}$	$\frac{1}{h^2}\,\mathbf{z}^\top\mathbf{M}$
$\mathbf{v}^-$	$\frac{1}{h}\mathbf{M}$	$\mathbf{0}$	$\frac{1}{h}\,\mathbf{z}^\top\mathbf{M}$
k_e	$\mathbf{G}_e^\top\mathbf{p}_e$	$\mathbf{G}_e^\top\mathbf{G}_e$	$(\mathbf{G}_e\mathbf{z})^\top\left(\mathbf{p}_e - \mathbf{G}_e\mathbf{x}^+\right)$
l_{rest}	$k_e\,\mathbf{G}_e^\top\hat{\mathbf{e}}$	$\mathbf{0}$	$k_e\,(\mathbf{G}_e\mathbf{z})^\top\hat{\mathbf{e}}$

Over a trajectory, a parameter collects one contribution per step, using the adjoint vectors already computed by the backward pass:

$$\frac{d\,\phi}{d\,\theta} = \sum_{t=1}^{T} \mathbf{z}_t^\top \left(\frac{\partial\,\mathbf{b}_t}{\partial\,\theta} - \frac{\partial\,\mathbf{L}_{\mathrm{PD}}}{\partial\,\theta}\,\mathbf{x}_t^+ \right)$$

Collision Handling Methods

Penalty / Constraint Based

Projective Dynamics: Fusing Constraint Projections for Fast Simulation



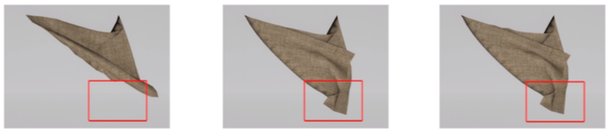
Barrier-Based

Penetration-free Projective Dynamics on the GPU



After Elastic Solve Collision Resolution

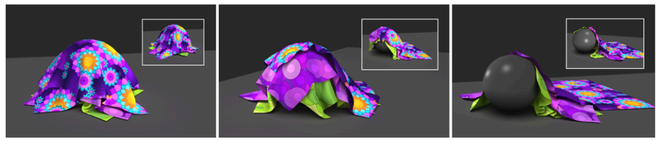
Differentiable Cloth Simulation for Inverse Problems



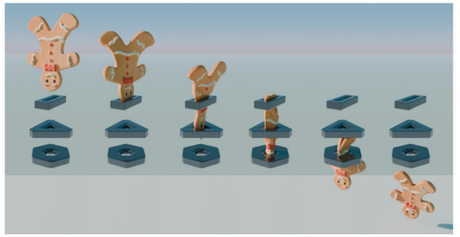
Initial guess | Optimized result | Target

Complementarity Coulomb/Signorini Based

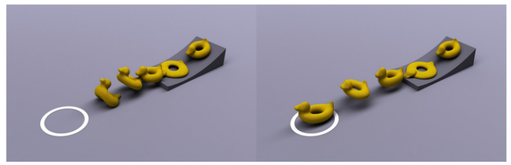
Projective Dynamics with Dry Frictional Contact



Fast But Accurate: A Real-Time Hyperelastic Simulator with Robust Frictional Contact



DiffPD: Differentiable Projective Dynamics



Collision Handling Methods

After Elastic Solve Collision Resolution

Projective Dynamics: Fusing Constraint Projections for Fast Simulation



After Elastic Solve Collision Resolution

Differentiable Cloth Simulation for Inverse Problems

Coulomb/Signorini Conditions

Projective Dynamics with Dry Frictional Contact

DiffPD: Differentiable Projective Dynamics

Fast But Accurate: A Real-Time Hyperelastic Simulator with Robust Frictional Contact

Barrier-Based

Penetration-free Projective Dynamics on the GPU

Penalty / Constraint Based

Projective Dynamics: Fusing Constraint Projections for Fast Simulation



- Collisions are **unilateral constraints**: a vertex may not cross a plane.
- Type: **vertex against plane**.
- Each collision constraint has a weight. Because hard constraints are approximated by high weights, **the collision acts like a spring** that pushes the colliding vertex out of the contact.
- Each new or removed collision **changes the global matrix**.
- Those changes are handled with fast rank updates and downdates of the prefactorized Cholesky factor, not a full refactorization.
- **Friction and restitution are applied afterwards** by changing the velocities of colliding vertices.

Collision Handling Methods

After Elastic Solve Collision Resolution

Projective Dynamics: Fusing Constraint Projections for Fast Simulation

After Elastic Solve Collision Resolution

Differentiable Cloth Simulation for Inverse Problems



Coulomb/Signorini Conditions

Projective Dynamics with Dry Frictional Contact

Fast But Accurate: A Real-Time Hyperelastic Simulator with Robust Frictional Contact

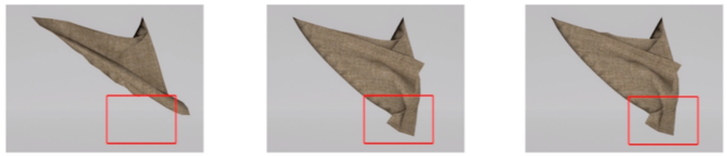
DiffPD: Differentiable Projective Dynamics

Barrier-Based

Penetration-free Projective Dynamics on the GPU

After Elastic Solve Collision Resolution

Differentiable Cloth Simulation for Inverse Problems



- Type: **vertex-face and edge-edge**.
- A pair that comes within the cloth thickness during the step gets a linear constraint: the signed distance must stay at least the thickness, written as $G\,z + h \leq 0$.
- The **collision response** runs **after the elastic solve**. It is an **impact-zone quadratic program** (QP): find the positions z closest to the current positions x, weighted by the vertex masses, subject to the linear constraints.
- Indipendent of the elastic solver.
- No spring or penalty is involved. The constraints are hard and the response is a minimal-displacement projection.
- **Friction is not handled** here (but could in theory??).
- For backpropagation: the gradient is computed with implicit differentiation of the KKT conditions of that QP, rather than by differentiating the response directly.

Collision Handling Methods

After Elastic Solve Collision Resolution

Projective Dynamics: Fusing Constraint Projections for Fast Simulation

After Elastic Solve Collision Resolution

Differentiable Cloth Simulation for Inverse Problems

Coulomb/Signorini Conditions

Projective Dynamics with Dry Frictional Contact

DiffPD: Differentiable Projective Dynamics



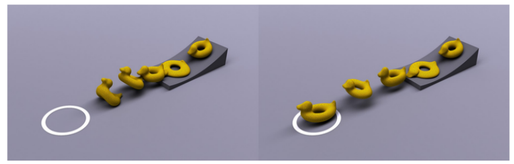
Fast But Accurate: A Real-Time Hyperelastic Simulator with Robust Frictional Contact

Barrier-Based

Penetration-free Projective Dynamics on the GPU

Coulomb/Signorini Conditions

DiffPD: Differentiable Projective Dynamics



- Contact is nodal and unilateral (**Signorini-type**): each node either stays off the obstacle with zero contact force, or touches it with a non-negative normal force.
- Type: **node against a signed-distance surface**.
- Each time step, collision detection proposes a candidate contact set C and a target point on the surface for each node in C (the paper uses CCD).
- Inside the step, the local-global loop runs with C fixed. The **global step solves only for the free nodes**, with the **contact nodes held at their targets**, which models a sticky surface and **gives static friction**. **Sliding friction is not modelled**.
- **The global matrix changes when C changes**. The rows and columns of contact nodes are removed, leaving the free-node submatrix.
- After the inner loop converges, the contact forces are recovered from the residual. Nodes whose normal force is negative are removed from C, and nodes that now penetrate the surface are added. Then a new outer iteration starts with the updated C and new targets.

Collision Handling Methods

After Elastic Solve Collision Resolution

Projective Dynamics: Fusing Constraint Projections for Fast Simulation

After Elastic Solve Collision Resolution

Differentiable Cloth Simulation for Inverse Problems

Coulomb/Signorini Conditions

Projective Dynamics with Dry Frictional Contact



DiffPD: Differentiable Projective Dynamics

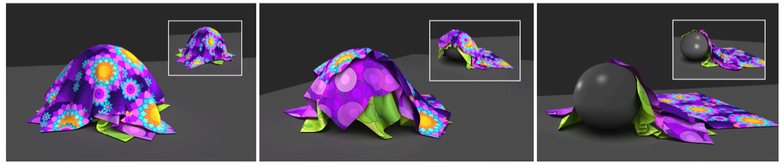
Fast But Accurate: A Real-Time Hyperelastic Simulator with Robust Frictional Contact

Barrier-Based

Penetration-free Projective Dynamics on the GPU

Coulomb/Signorini Conditions

Projective Dynamics with Dry Frictional Contact



- **Contacts are nodal**: a particle either leaves the obstacle or touches it.
- Type: **node against collider/primitives**.
- The contact rules are **Signorini-Coulomb**: a touching particle can only push away from the surface, and friction is limited by the normal force.
- Each contact has three cases:
  - **take-off** (the particle lifts off),
  - **stick** (no sliding)
  - **slip** (it slides, with friction opposing the motion).
- **The big global matrix never changes**. Contact forces are updated locally, particle by particle, inside each iteration, so the **global factorization is reused and never rebuilt**.
- Self-collisions are handled by the same rules applied between pairs of particles on the same cloth.
- **Both static and sliding friction are supported**.

Collision Handling Methods

After Elastic Solve Collision Resolution

Projective Dynamics: Fusing Constraint Projections for Fast Simulation

After Elastic Solve Collision Resolution

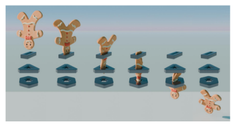
Differentiable Cloth Simulation for Inverse Problems

Coulomb/Signorini Conditions

Projective Dynamics with Dry Frictional Contact

DiffPD: Differentiable Projective Dynamics

Fast But Accurate: A Real-Time Hyperelastic Simulator with Robust Frictional Contact

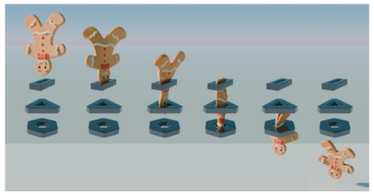


Barrier-Based

Penetration-free Projective Dynamics on the GPU

Coulomb/Signorini Conditions

Fast But Accurate: A Real-Time Hyperelastic Simulator with Robust Frictional Contact



- Built for fast, stable real-time simulation of stiff elastic objects.
- Contacts follow the **Signorini-Coulomb rules**: a contact can only push, and friction is limited by how hard the surfaces press together.
- The rules are written as **complementarity conditions**.
- The global solve is done with a special root-finding method (non-smooth Newton) that works on those conditions directly.
- The big **global system matrix stays the same throughout the simulation**, so it is **factorized once at the start**.
- The paper's main trick: it precomputes the inverse of that matrix in sparse form, so each global solve becomes two sparse matrix-vector multiplications. These parallelize well on a GPU. **It means PD global step can be don eon the GPU**.
- It also supports binding constraints (for example joints) and works with hyperelastic materials, including stiff ones.

Collision Handling Methods

After Elastic Solve Collision Resolution

Projective Dynamics: Fusing Constraint Projections for Fast Simulation

After Elastic Solve Collision Resolution

Differentiable Cloth Simulation for Inverse Problems

Coulomb/Signorini Conditions

Projective Dynamics with Dry Frictional Contact

Fast But Accurate: A Real-Time Hyperelastic Simulator with Robust Frictional Contact

DiffPD: Differentiable Projective Dynamics

Barrier-Based

Penetration-free Projective Dynamics on the GPU



Barrier-Based

Penetration-free Projective Dynamics on the GPU



Collision Handling Methods

Penalty / Constraint Based

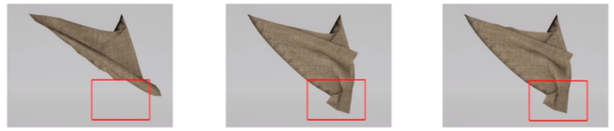
Projective Dynamics: Fusing Constraint Projections for Fast Simulation

Barrier-Based

Penetration-free Projective Dynamics on the GPU

After Elastic Solve Collision Resolution

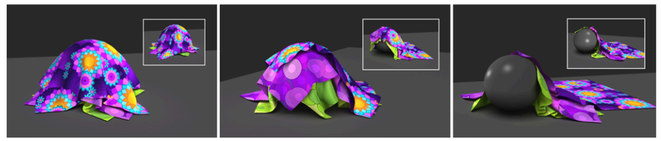
Differentiable Cloth Simulation for Inverse Problems



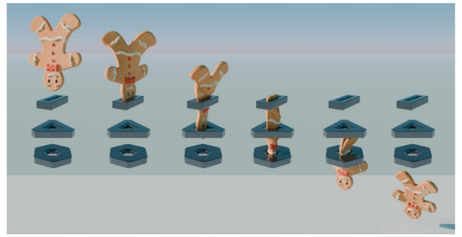
Initial guess | Optimized result | Target

Coulomb/Signorini Conditions

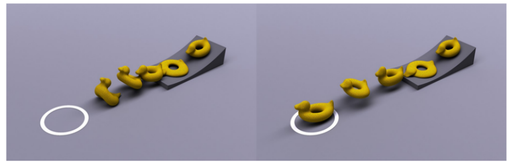
Projective Dynamics with Dry Frictional Contact



Fast But Accurate: A Real-Time Hyperelastic Simulator with Robust Frictional Contact



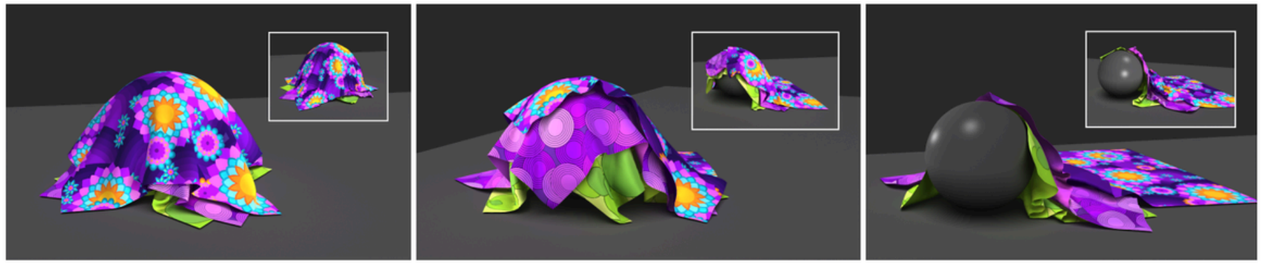
DiffPD: Differentiable Projective Dynamics



Projective Dynamics with Dry Frictional Contact

Projective Dynamics with Dry Frictional Contact

MICKAËL LY, Univ. Grenoble Alpes, Inria, CNRS, Grenoble INP, LJK, France
JEAN JOUVE, ENS Rennes and Univ. Grenoble Alpes, Inria, CNRS, Grenoble INP, LJK, France
LAURENCE BOISSIEUX, Univ. Grenoble Alpes, Inria, France
FLORENCE BERTAILS-DESCOUBES, Univ. Grenoble Alpes, Inria, CNRS, Grenoble INP, LJK, France



[https://dl.acm.org/doi/10.1145/3386569.3392396]

Velocity Based Projective Dynamics

Since Signorini-Coulomb Law for contact with friction are writte using velocity, first we derive the Velocity-Based PD variant. We start from the Position-Based PD global step:

$$\mathbf{L}\mathbf{x}^{+} = \mathbf{b} \quad \text{with} \quad \begin{aligned} \mathbf{L} &= \mathbf{M} + h^2 \sum k_e \mathbf{G}_e^{\top} \mathbf{G}_e \\ \mathbf{b} &= \mathbf{M}\tilde{\mathbf{x}} + h^2 \sum k_e \mathbf{G}_e^{\top} \mathbf{p}_e \end{aligned}$$

We substitute $\mathbf{x}^{+} = (\mathbf{x}^{-} + h\mathbf{v}^{+})$:

$$\mathbf{L}\left(\mathbf{x}^{-} + h\mathbf{v}^{+}\right) = \mathbf{b} \quad \Rightarrow \quad \mathbf{L}\mathbf{v}^{+} = \underbrace{\frac{1}{h}\left(\mathbf{b} - \mathbf{L}\mathbf{x}^{-}\right)}_{\tilde{\mathbf{b}}}$$

So that we obtain the Velocity-Based global step:

$$\mathbf{L}\mathbf{v}^{+} = \tilde{\mathbf{b}}$$

The system matrix is the same as in the Position-Based variant, the only change is in the computation of the right hand side, where we subtract the factor $\mathbf{L}\mathbf{x}^{-}$ and divide by h

Velocity Based Projective Dynamics

function VELOCITYPROJECTIVEDYNAMICS($\mathbf{x}^-, \mathbf{v}^-$)

$\tilde{\mathbf{x}} \leftarrow \mathbf{x}^- + h\,\mathbf{v}^-$

Initialize $\mathbf{v}^{(0)}$

$\mathbf{x}^{(0)} \leftarrow \mathbf{x}^- + h\,\mathbf{v}^{(0)}$

for $k = 0, 1, \ldots, N-1$ **do**

  for each element e **in parallel do**

    $\mathbf{p}_e \leftarrow \text{PROJ}_{\mathcal{M}_e}\left(\mathbf{G}_e\,\mathbf{x}^{(k)}\right)$

  $\tilde{\mathbf{b}} \leftarrow \frac{1}{h}\left(\mathbf{M}\,\tilde{\mathbf{x}} + h^2 \sum_e k_e\,\mathbf{G}_e^\top \mathbf{p}_e - \mathbf{L}\,\mathbf{x}^-\right)$

  SOLVE $\mathbf{L}\,\mathbf{v}^{(k+1)} = \tilde{\mathbf{b}}$

  $\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^- + h\,\mathbf{v}^{(k+1)}$

$\mathbf{v}^+ \leftarrow \mathbf{v}^{(N)}$

$\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$

end function

PD with Dry Frictional Contact

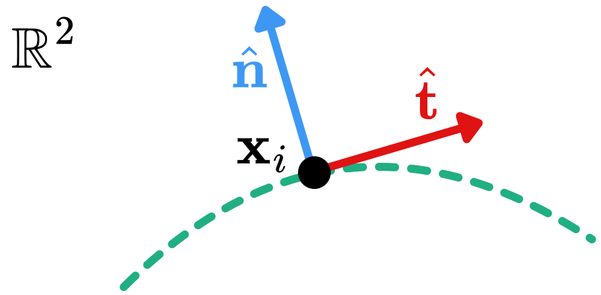
Contact Frame

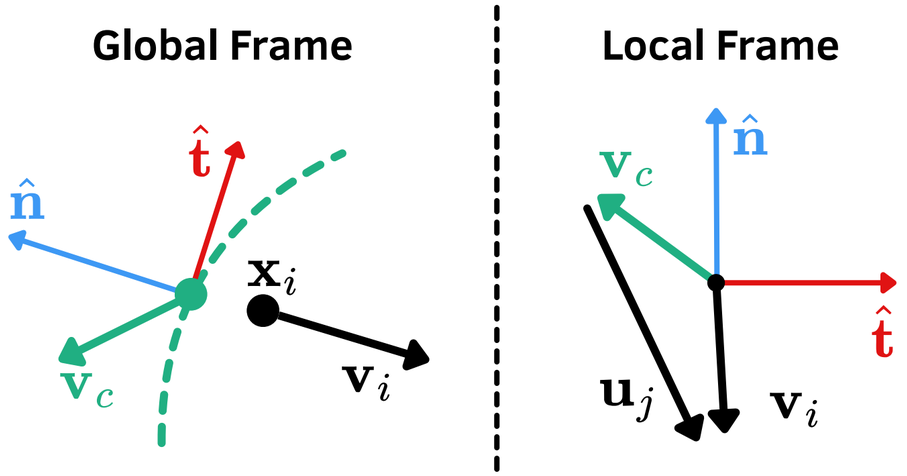
j : contact index i : colliding particle index

$\mathbf{R}_j = [\,\hat{\mathbf{n}}_j\ \hat{\mathbf{t}}_j\ \hat{\mathbf{t}}'_j\,]$: **local frame**, which acts as a rotation matrix from local contact space to world contact. $\mathbf{R}_j^\top$ does the opposite.

$\mathbf{u}_j = \mathbf{R}_j^\top(\mathbf{v}_i - \mathbf{v}_c)$: **relative velocity**, local frame velocity difference between particle and the obstacle's velocity at the contact point

$\mathbf{r}_j \in \mathbb{R}^3$: **contact impulse,** applied to the particle by the obstacle, in contact-frame coordinates





PD with Dry Frictional Contact

Signorini-Coulomb Law

For each contact the Signorini–Coulomb law constrains
the relative velocity and local contact impulse to satisfy:

$$(\mathbf{r}_j, \mathbf{u}_j) \in \mathcal{C}_{\mu_j} \qquad \text{with} \;\; \mu_j \geq 0 \;\; \text{: friction coeffiecient}$$

Defined by three mutually exclusive and exhaustive cases:

- take-off: $\mathbf{r}_j = \mathbf{0}, \qquad u_{j,\mathrm{N}} \geq 0$
- stick: $\|\mathbf{r}_{j,\mathrm{T}}\| \leq \mu_j \, r_{j,\mathrm{N}}, \quad \mathbf{u}_j = \mathbf{0}$
- slip: $\|\mathbf{r}_{j,\mathrm{T}}\| = \mu_j \, r_{j,\mathrm{N}}, \quad u_{j,\mathrm{N}} = 0, \;\; \mathbf{r}_{j,\mathrm{T}} = -\alpha \, \mathbf{u}_{j,\mathrm{T}}, \;\; \alpha > 0$

PD with Dry Frictional Contact

Signorini-Coulomb Law

For each contact the Signorini–Coulomb law constrains
the relative velocity and local contact impulse to satisfy:

$$(\mathbf{r}_j, \mathbf{u}_j) \in \mathcal{C}_{\mu_j} \qquad \text{with} \;\; \mu_j = 0 \;\; \text{: no friction}$$

Defined by two mutually exclusive and exhaustive cases:

- take-off: $\mathbf{r}_j = \mathbf{0}, \qquad u_{j,\mathrm{N}} \geq 0$
- contact: $\mathbf{r}_j = (r_{j,\mathrm{N}}, \mathbf{0}), \;\; r_{j,\mathrm{N}} \geq 0, \;\; u_{j,\mathrm{N}} = 0, \;\; \mathbf{u}_{j,\mathrm{T}}$ free

PD with Dry Frictional Contact

Adding contact/frictional impulse to the Velocity-Based PD framework yields:

$$\begin{cases} \mathbf{L}\,\mathbf{v}^{+} = \tilde{\mathbf{b}} + \mathbf{J}_c^{\top}\mathbf{r}^{+} \\ \mathbf{u}^{+} = \mathbf{J}_c\left(\mathbf{v}^{+} - \mathbf{v}_c\right) \\ \left(\mathbf{r}_j^{+},\, \mathbf{u}_j^{+}\right) \in \mathcal{C}_{\mu_j} \qquad \forall\, j = 1, \ldots, N_c \end{cases}$$

$\mathbf{J}_c \in \mathbb{R}^{3N_c \times 3n}$: **contact Jacobian**, a sparse block matrix with one row of blocks per contact, each holding only the transposed local frame at its particle's column:

$$\text{world} \;\rightarrow \mathbf{J}_c \rightarrow\; \text{contact}$$

$$\text{contact} \;\rightarrow \mathbf{J}_c^{\top} \rightarrow \text{world}$$

PD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \underbrace{h^2 \sum k_e \mathbf{G}_e^\top \mathbf{G}_e}_{\mathbf{C}} = \mathbf{M} + \mathbf{C}$$

Positive Diagonal

Positive Semi Definite

Using a Jacobi-like split, treating $\mathbf{M}$ implicitly and lagging $\mathbf{C}$ to the previous iterate, the global equation becomes:

$$\mathbf{M}\mathbf{v}^{(k+1)} = \underbrace{\tilde{\mathbf{b}} - \mathbf{C}\mathbf{v}^{(k)}}_{\mathbf{f}} + \mathbf{J}_c^\top \mathbf{r}^{(k+1)} = \mathbf{f} + \mathbf{J}_c^\top \mathbf{r}^{(k+1)}$$

The vector $\mathbf{f}$ is fully computable from previous-iteration data.
Exploiting the diagonal structure of the mass matrix, we write the per-node equation:

$$m_i \, \mathbf{v}_i^{(k+1)} = \begin{cases} \mathbf{f}_i & \text{if } i \text{ not in contact} \\ \mathbf{f}_i + \mathbf{R}_j \, \mathbf{r}_j^{(k+1)} & \text{if } i \text{ in contact } j \end{cases}$$

PD with Dry Frictional Contact

Using the fact: $\mathbf{u}_j = \mathbf{R}_j^\top \left(\mathbf{v}_i - \mathbf{v}_c \right)$ we obtain:

$$m_i \, \mathbf{u}_j = \underbrace{\mathbf{R}_j^\top \left(\mathbf{f}_i - m_i \, \mathbf{v}_c \right)}_{\mathbf{d}_j \text{ : fully known}} + \mathbf{r}_j \qquad \left(\mathbf{r}_j, \, \mathbf{u}_j \right) \in \mathcal{C}_{\mu_j \geq 0}$$

↳ with friction

$\mathbf{d}_j$ is the relative velocity (times mass) the node would have with no contact.
Choose $\mathbf{r}_j$ so that $m_i \mathbf{u}_j = \mathbf{d}_j + \mathbf{r}_j$ satisfies Signorini–Coulomb:

$$\mathbf{r}_j = \begin{cases} \text{take-off:} & \mathbf{0} & \text{if } d_{j,\mathrm{N}} \geq 0 \\ \text{stick:} & -\mathbf{d}_j & \text{if } \left\| \mathbf{d}_{j,\mathrm{T}} \right\| \leq -\mu_j \, d_{j,\mathrm{N}} \\ \text{slip:} & \left(-d_{j,\mathrm{N}}, \;\; \mu_j \, d_{j,\mathrm{N}} \, \dfrac{\mathbf{d}_{j,\mathrm{T}}}{\left\| \mathbf{d}_{j,\mathrm{T}} \right\|} \right) & \text{otherwise} \end{cases}$$

PD with Dry Frictional Contact

Using the fact: $\mathbf{u}_j = \mathbf{R}_j^\top \left(\mathbf{v}_i - \mathbf{v}_c \right)$ we obtain:

$$m_i \, \mathbf{u}_j = \underbrace{\mathbf{R}_j^\top \left(\mathbf{f}_i - m_i \, \mathbf{v}_c \right)}_{\mathbf{d}_j \text{ : fully known}} + \mathbf{r}_j \qquad \left(\mathbf{r}_j, \, \mathbf{u}_j \right) \in \mathcal{C}_{\mu_j = 0}$$

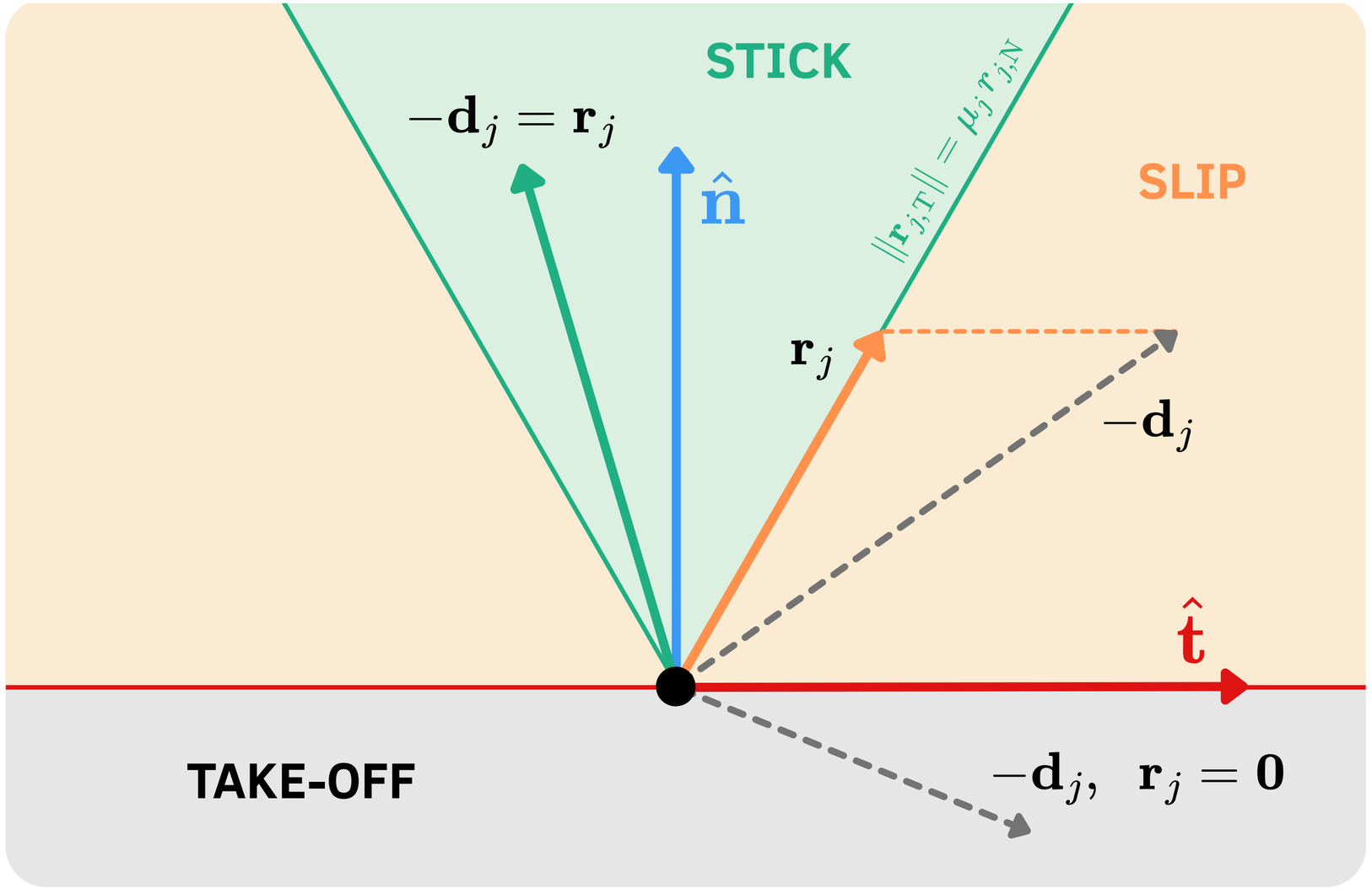
↳ without friction

$\mathbf{d}_j$ is the relative velocity (times mass) the node would have with no contact.
Choose $\mathbf{r}_j$ so that $m_i \mathbf{u}_j = \mathbf{d}_j + \mathbf{r}_j$ satisfies Signorini:

$$\mathbf{r}_j = \begin{cases} \text{take-off:} \quad \mathbf{0} & \text{if } d_{j,\mathrm{N}} \geq 0 \\ \text{contact:} \quad (-d_{j,\mathrm{N}}, \;\; \mathbf{0}) & \text{otherwise} \end{cases}$$

PD with Dry Frictional Contact

Friction Cone



PD with Dry Frictional Contact

$$\Phi(\mathbf{f}) = \big[\, \Phi_1, \ldots, \Phi_n \,\big], \qquad \Phi_i(\mathbf{f}) = \begin{cases} \mathbf{r}_i(\mathbf{d}_i) & \text{particle } i \text{ in contact} \\ \mathbf{0} & \text{otherwise} \end{cases}$$

$$\mathbf{d}_i = \mathbf{f}_i - m_i\,\mathbf{v}_c, \quad d_\mathrm{N} = \hat{\mathbf{n}}_i^\top \mathbf{d}_i, \quad \mathbf{d}_\mathrm{T} = \mathbf{d}_i - d_\mathrm{N}\,\hat{\mathbf{n}}_i, \quad \hat{\mathbf{d}}_\mathrm{T} = \frac{\mathbf{d}_\mathrm{T}}{\|\mathbf{d}_\mathrm{T}\|}$$

With Friction:

function UPDATECONTACTFORCE($\mathbf{f}_i, m_i, \hat{\mathbf{n}}_i, \mathbf{v}_c, \mu$)

$\mathbf{d}_i \leftarrow \mathbf{f}_i - m_i\,\mathbf{v}_c$

$d_\mathrm{N} \leftarrow \hat{\mathbf{n}}_i^\top \mathbf{d}_i, \quad \mathbf{d}_\mathrm{T} \leftarrow \mathbf{d}_i - d_\mathrm{N}\,\hat{\mathbf{n}}_i$

if $d_\mathrm{N} \geq 0$ **then**

  return $\mathbf{0}$ ▷ take-off

else if $\|\mathbf{d}_\mathrm{T}\| \leq -\mu\, d_\mathrm{N}$ **then**

  return $-\mathbf{d}_i$ ▷ stick

else

  return $-d_\mathrm{N}\,\hat{\mathbf{n}}_i + \mu\, d_\mathrm{N}\,\hat{\mathbf{d}}_\mathrm{T}$ ▷ slip

end function

Friction-less:

function UPDATECONTACTFORCE($\mathbf{f}_i, m_i, \hat{\mathbf{n}}_i, \mathbf{v}_c$)

$d_\mathrm{N} \leftarrow \hat{\mathbf{n}}_i^\top \left(\mathbf{f}_i - m_i\,\mathbf{v}_c\right)$

if $d_\mathrm{N} \geq 0$ **then**

  return $\mathbf{0}$ ▷ take-off

else

  return $-d_\mathrm{N}\,\hat{\mathbf{n}}_i$ ▷ contact

end function

$$\mathbf{r}_i = \begin{cases} \mathbf{0} & d_\mathrm{N} \geq 0 \\ -\mathbf{P}_i\,\mathbf{d}_i & d_\mathrm{N} < 0 \;\; \text{(active)} \end{cases} \qquad \mathbf{P}_i = \hat{\mathbf{n}}_i \hat{\mathbf{n}}_i^\top$$

PD with Dry Frictional Contact

function PDFrictionalContactStep($\mathbf{x}^-, \mathbf{v}^-$)

$\tilde{\mathbf{x}} \leftarrow \mathbf{x}^- + h\,\mathbf{v}^- + h^2\mathbf{M}^{-1}\mathbf{f}_{\text{ext}}$

$\{i, \hat{\mathbf{n}}_i, \mathbf{v}_c\}_i \leftarrow$ DetectContacts($\tilde{\mathbf{x}}$)

$\mathbf{x}^{(0)} \leftarrow \tilde{\mathbf{x}}, \quad \mathbf{v}^{(0)} \leftarrow \mathbf{v}^-$

for $k = 0, 1, \ldots, N-1$ **do**

for each element e **in parallel do**

$\mathbf{p}_e \leftarrow \text{Proj}_{\mathcal{M}_e}(\mathbf{G}_e\mathbf{x}^{(k)})$

$\tilde{\mathbf{b}} \leftarrow \frac{1}{h}\left(\mathbf{M}\,\tilde{\mathbf{x}} + h^2 \sum_e k_e\,\mathbf{G}_e^\top \mathbf{p}_e - \mathbf{L}\,\mathbf{x}^-\right)$

$\mathbf{f} \leftarrow \tilde{\mathbf{b}} - \mathbf{C}\,\mathbf{v}^{(k)}$

for each contact particle i **in parallel do**

$\mathbf{r}_i \leftarrow$ UpdateContactForce($\mathbf{f}_i, m_i, \hat{\mathbf{n}}_i, \mathbf{v}_c, \mu$)

$\tilde{\mathbf{b}}_i \mathrel{+}= \mathbf{r}_i$

Solve $\mathbf{L}\,\mathbf{v}^{(k+1)} = \tilde{\mathbf{b}}$

$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^- + h\,\mathbf{v}^{(k+1)}$

$\mathbf{v}^+ \leftarrow \mathbf{v}^{(N)}$

$\mathbf{x}^+ \leftarrow \mathbf{x}^{(N)}$

end function

Collision Detection

Find the particles that will hit an obstacle, using the predicted positions. Each contact gets its frame and the obstacle velocity

Local step: Constraints

Project each element onto its rest shape, as in standard PD, then build the right-hand side from these projections.

Local Step: Collisions

Estimate each contacting particle's momentum without contact. Choose the impulse that satisfies Signorini–Coulomb, then add it to the right-hand side

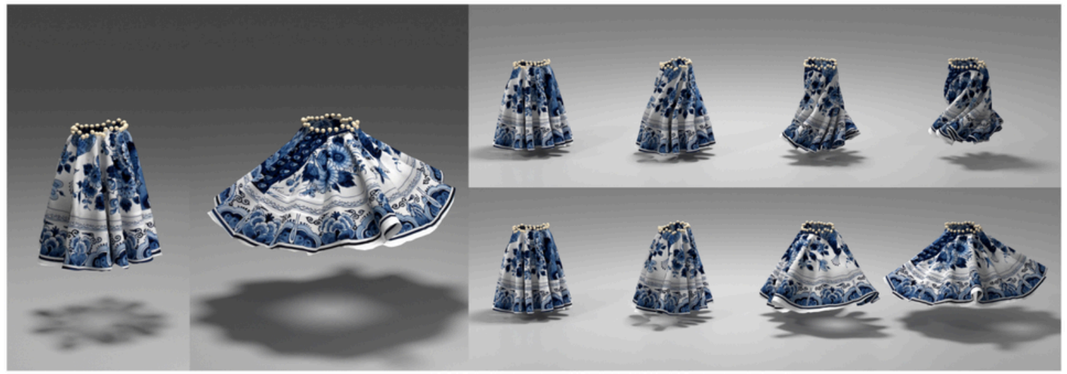
Global Step

One solve with the prefactored System Matrix. Contact changes only the right-hand side.

Differentiable PD with Dry Frictional Contact

DiffCloth: Differentiable Cloth Simulation with Dry Frictional Contact

YIFEI LI and TAO DU, MIT CSAIL, USA
KUI WU, Tencent Lightspeed & Quantum Studios, USA
JIE XU and WOJCIECH MATUSIK, MIT CSAIL, USA



https://people.csail.mit.edu/liyifei/publication/diffcloth/

DiffPD with Dry Frictional Contact

Position-PD

$$\mathbf{L} = \frac{1}{h^2}\mathbf{M} + \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e$$

$$\mathbf{b} = \frac{1}{h^2}\mathbf{M}\tilde{\mathbf{x}} + \sum k_e \, \mathbf{G}_e^\top \mathbf{p}_e$$

At convergence it holds:

$$\mathbf{L}(\theta)\, \mathbf{x}^+ = \mathbf{b}(\theta, \mathbf{x}^+)$$

Steps to compute Gradient:

- Find **Real Hessian**:

$$\mathbf{H}_{\text{PD}} = \mathbf{L} - \Delta\mathbf{L}$$

- Find method to compute **Adjoint Vector**:

$$\mathbf{L}\, \mathbf{z}^{(k+1)} = \Delta\mathbf{L}\, \mathbf{z}^{(k)} + \left(\frac{\partial \phi}{\partial \mathbf{x}^+}\right)^\top$$

- Find formula to compute **Gradient**:

$$\frac{d\,\phi}{d\,\theta} = \mathbf{z}^\top \left(\frac{\partial\, \mathbf{b}}{\partial\, \theta} - \frac{\partial\, \mathbf{L}}{\partial\, \theta}\, \mathbf{x}^+\right)$$

Velocity-PD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C} \qquad \mathbf{C} = h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M}\tilde{\mathbf{x}} + h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{p}_e \qquad \tilde{\mathbf{b}} = \frac{1}{h}\left(\mathbf{b} - \mathbf{L}\, \mathbf{x}^-\right)$$

$$\mathbf{r} = \Phi(\mathbf{f}) \qquad \mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C}\, \mathbf{v}^+$$

At convergence it holds:

$$\mathbf{L}(\theta)\mathbf{v}^+ = \tilde{\mathbf{b}}(\theta, \mathbf{v}^+) + \mathbf{r}(\theta, \mathbf{v}^+)$$

Steps to compute Gradient:

- Find **Real Hessian**
- Find method to compute **Adjoint Vector**
- Find formula to compute **Gradient**

DiffPD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C} \qquad \mathbf{C} = h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e \qquad \Delta\mathbf{L} = \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \, \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M}\tilde{\mathbf{x}} + h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{p}_e \qquad \tilde{\mathbf{b}} = \tfrac{1}{h}\left(\mathbf{b} - \mathbf{L}\,\mathbf{x}^-\right) \qquad \boldsymbol{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial\left(\mathbf{G}_e \mathbf{x}\right)}$$

$$\mathbf{r} = \Phi(\mathbf{f}) \qquad \mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C}\,\mathbf{v}^+ \qquad \boldsymbol{\Gamma}_{\mathbb{C}} = \frac{\partial \Phi}{\partial \mathbf{f}}$$

- Find **Real Hessian**:

$$\mathbf{H}_c = \mathbf{M} + h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e \;-\; h^2 \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \mathbf{G}_e \;-\; \boldsymbol{\Gamma}_c \left(h^2 \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \mathbf{G}_e - h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e \right)$$

DiffPD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C} \qquad \mathbf{C} = h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e \qquad \Delta\mathbf{L} = \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \, \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M}\tilde{\mathbf{x}} + h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{p}_e \qquad \tilde{\mathbf{b}} = \tfrac{1}{h}\left(\mathbf{b} - \mathbf{L}\,\mathbf{x}^-\right) \qquad \boldsymbol{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial\left(\mathbf{G}_e \mathbf{x}\right)}$$

$$\mathbf{r} = \Phi(\mathbf{f}) \qquad \mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C}\,\mathbf{v}^+ \qquad \boldsymbol{\Gamma}_{\mathbb{C}} = \frac{\partial \Phi}{\partial \mathbf{f}}$$

- Find **Real Hessian**:

$$\mathbf{H}_c = \overbrace{\mathbf{M} + h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e}^{\mathbf{L}} - \overbrace{h^2 \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \mathbf{G}_e}^{\Delta\mathbf{L}_c} - \overbrace{\boldsymbol{\Gamma}_c \left(h^2 \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \mathbf{G}_e - h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e\right)}^{\Delta\mathbf{R}^\top}$$

$$\mathbf{H}_c = \mathbf{L} - \Delta\mathbf{L}_c - \Delta\mathbf{R}^\top \qquad \text{with} \quad \begin{aligned} \Delta\mathbf{L}_c &= h^2 \Delta\mathbf{L} \\ \Delta\mathbf{R} &= \left(\Delta\mathbf{L}_c - \mathbf{C}\right) \boldsymbol{\Gamma}_c^\top \end{aligned}$$

DiffPD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C} \qquad \mathbf{C} = h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e \qquad \Delta\mathbf{L} = \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \, \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M}\tilde{\mathbf{x}} + h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{p}_e \qquad \tilde{\mathbf{b}} = \tfrac{1}{h}\left(\mathbf{b} - \mathbf{L}\,\mathbf{x}^-\right) \qquad \boldsymbol{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial \left(\mathbf{G}_e \mathbf{x}\right)}$$

$$\mathbf{r} = \Phi(\mathbf{f}) \qquad \mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C}\,\mathbf{v}^+ \qquad \boldsymbol{\Gamma}_{\mathbb{C}} = \frac{\partial \Phi}{\partial \mathbf{f}}$$

- Find **Real Hessian**:

$$\mathbf{H}_c = \overbrace{\mathbf{M} + h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e}^{\mathbf{L}} - \overbrace{h^2 \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \mathbf{G}_e}^{\Delta\mathbf{L}_c} - \overbrace{\boldsymbol{\Gamma}_c \left(h^2 \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \mathbf{G}_e - h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e\right)}^{\Delta\mathbf{R}^\top}$$

$$\mathbf{H}_c = \mathbf{L} - \Delta\mathbf{L}_c - \Delta\mathbf{R}^\top \qquad \text{with} \quad \begin{aligned} \Delta\mathbf{L}_c &= h^2 \Delta\mathbf{L} \\ \Delta\mathbf{R} &= \left(\Delta\mathbf{L}_c - \mathbf{C}\right) \boldsymbol{\Gamma}_c^\top \end{aligned}$$

- Find method to compute **Adjoint Vector**:

$$\left(\mathbf{L} - \Delta\mathbf{L}_c - \Delta\mathbf{R}\right) \mathbf{z} = \left(\frac{\partial \phi}{\partial \mathbf{v}^+}\right)^\top \qquad \Rightarrow$$

Iterative Solver

$$\mathbf{L}\,\mathbf{z}^{(k+1)} = \left(\Delta\mathbf{L}_c + \Delta\mathbf{R}\right) \mathbf{z}^{(k)} + \left(\frac{\partial \phi}{\partial \mathbf{v}^+}\right)^\top$$

DiffPD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C} \qquad \mathbf{C} = h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e \qquad \Delta\mathbf{L} = \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \, \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M}\tilde{\mathbf{x}} + h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{p}_e \qquad \tilde{\mathbf{b}} = \tfrac{1}{h}\left(\mathbf{b} - \mathbf{L}\,\mathbf{x}^-\right) \qquad \boldsymbol{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial\left(\mathbf{G}_e \mathbf{x}\right)}$$

$$\mathbf{r} = \Phi(\mathbf{f}) \qquad \mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C}\,\mathbf{v}^+ \qquad \boldsymbol{\Gamma}_{\mathbb{C}} = \frac{\partial \Phi}{\partial \mathbf{f}}$$

Adjoint Vector Iterative Solver:

$$\mathbf{L}\,\mathbf{z}^{(k+1)} = h^2 \Delta\mathbf{L}\,\bar{\mathbf{z}}^{(k)} + \mathbf{C}\left(\mathbf{z}^{(k)} - \bar{\mathbf{z}}^{(k)}\right) + \left(\frac{\partial \phi}{\partial \mathbf{v}^+} + h\,\frac{\partial \phi}{\partial \mathbf{x}^+}\right)^\top \qquad \bar{\mathbf{z}} = \left(\mathbf{I} + \boldsymbol{\Gamma}_{\mathbb{C}}\right)^\top \mathbf{z}$$

DiffPD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C} \qquad \mathbf{C} = h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e \qquad \Delta\mathbf{L} = \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \, \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M}\tilde{\mathbf{x}} + h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{p}_e \qquad \tilde{\mathbf{b}} = \tfrac{1}{h}\left(\mathbf{b} - \mathbf{L}\,\mathbf{x}^-\right) \qquad \boldsymbol{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial\left(\mathbf{G}_e \mathbf{x}\right)}$$

$$\mathbf{r} = \Phi(\mathbf{f}) \qquad \mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C}\,\mathbf{v}^+ \qquad \boldsymbol{\Gamma}_{\mathbb{C}} = \frac{\partial \Phi}{\partial \mathbf{f}}$$

Adjoint Vector Iterative Solver:

$$\mathbf{L}\,\mathbf{z}^{(k+1)} = \underbrace{h^2 \Delta\mathbf{L}\,\bar{\mathbf{z}}^{(k)}}_{\text{Local Elastic}} + \underbrace{\mathbf{C}\left(\mathbf{z}^{(k)} - \bar{\mathbf{z}}^{(k)}\right)}_{\text{Local Contact}} + \left(\frac{\partial \phi}{\partial \mathbf{v}^+} + h\,\frac{\partial \phi}{\partial \mathbf{x}^+}\right)^\top \qquad \bar{\mathbf{z}} = \left(\mathbf{I} + \boldsymbol{\Gamma}_{\mathbb{C}}\right)^\top \mathbf{z}$$

It is **0** for nodes not in contact.

The constant system matrix of the forward PD, already factorized: each iteration costs one back-substitution.

The loss usually depends on positions. Since $\mathbf{x}^+ = \mathbf{x}^- + h\mathbf{v}^+$, the position gradient reaches velocity scaled by h.

Filtered adjoint:
at contact nodes it removes the components the contact controls.

Same local/global structure as DiffPD: two parallel local steps and one global solve.
Without contact, $\bar{\mathbf{z}} = \mathbf{z}$, and we recover the DiffPD iteration.

DiffPD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C} \qquad \mathbf{C} = h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e \qquad \Delta\mathbf{L} = \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \, \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M}\tilde{\mathbf{x}} + h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{p}_e \qquad \tilde{\mathbf{b}} = \tfrac{1}{h}\left(\mathbf{b} - \mathbf{L}\,\mathbf{x}^-\right) \qquad \boldsymbol{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial\left(\mathbf{G}_e \mathbf{x}\right)}$$

$$\mathbf{r} = \Phi(\mathbf{f}) \qquad \mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C}\,\mathbf{v}^+ \qquad \boldsymbol{\Gamma}_{\mathbb{C}} = \frac{\partial \Phi}{\partial \mathbf{f}}$$

Adjoint Vector Iterative Solver:

$$\mathbf{L}\,\mathbf{z}^{(k+1)} = h^2 \Delta\mathbf{L}\,\bar{\mathbf{z}}^{(k)} + \mathbf{C}\left(\mathbf{z}^{(k)} - \bar{\mathbf{z}}^{(k)}\right) + \left(\frac{\partial \phi}{\partial \mathbf{v}^+} + h\,\frac{\partial \phi}{\partial \mathbf{x}^+}\right)^\top \qquad \bar{\mathbf{z}} = \left(\mathbf{I} + \boldsymbol{\Gamma}_{\mathbb{C}}\right)^\top \mathbf{z}$$

Formula for General Parameter Gradient:

$$\frac{d\,\phi}{d\,\theta} = \bar{\mathbf{z}}^\top \left(\frac{\partial \tilde{\mathbf{b}}}{\partial\,\theta} - \frac{\partial\,\mathbf{L}}{\partial\,\theta}\,\mathbf{v}^+\right)$$

DiffPD with Dry Frictional Contact

$$\mathbf{L} = \mathbf{M} + \mathbf{C} \qquad \mathbf{C} = h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{G}_e \qquad \Delta\mathbf{L} = \sum k_e \, \mathbf{G}_e^\top \boldsymbol{\Gamma}_e \, \mathbf{G}_e$$

$$\mathbf{b} = \mathbf{M}\tilde{\mathbf{x}} + h^2 \sum k_e \, \mathbf{G}_e^\top \mathbf{p}_e \qquad \tilde{\mathbf{b}} = \tfrac{1}{h}\left(\mathbf{b} - \mathbf{L}\,\mathbf{x}^-\right) \qquad \boldsymbol{\Gamma}_e = \frac{\partial \mathbf{p}_e}{\partial\left(\mathbf{G}_e \mathbf{x}\right)}$$

$$\mathbf{r} = \Phi(\mathbf{f}) \qquad \mathbf{f} = \tilde{\mathbf{b}} - \mathbf{C}\,\mathbf{v}^+ \qquad \boldsymbol{\Gamma}_{\mathbb{C}} = \frac{\partial \Phi}{\partial \mathbf{f}}$$

Adjoint Vector Iterative Solver:

$$\mathbf{L}\,\mathbf{z}^{(k+1)} = h^2 \Delta\mathbf{L}\,\bar{\mathbf{z}}^{(k)} + \mathbf{C}\left(\mathbf{z}^{(k)} - \bar{\mathbf{z}}^{(k)}\right) + \left(\frac{\partial \phi}{\partial \mathbf{v}^+} + h\,\frac{\partial \phi}{\partial \mathbf{x}^+}\right)^\top \qquad \bar{\mathbf{z}} = \left(\mathbf{I} + \boldsymbol{\Gamma}_{\mathbb{C}}\right)^\top \mathbf{z}$$

Formula for General Parameter Gradient:

$$\frac{d\,\phi}{d\,\theta} = \bar{\mathbf{z}}^\top \left(\frac{\partial \tilde{\mathbf{b}}}{\partial\,\theta} - \frac{\partial\,\mathbf{L}}{\partial\,\theta}\,\mathbf{v}^+\right)$$

Specialization:

Gradient of previous step Velocity:
$$\frac{d\,\phi}{d\,\mathbf{v}^-} = \bar{\mathbf{z}}^\top \mathbf{M}$$

Gradient of previous step Position:
$$\frac{d\,\phi}{d\,\mathbf{x}^-} = \bar{\mathbf{z}}^\top \left(h\,\Delta\mathbf{L} - \frac{1}{h}\,\mathbf{C}\right) + \frac{\partial\,\phi}{\partial\,\mathbf{x}^+}$$

Gradient of shared stiffness of all springs along all trajectory:
$$\frac{d\,\phi}{d\,k} = h \sum_{t=1}^{T} \sum_{e} \left(\bar{\mathbf{z}}_{t,i} - \bar{\mathbf{z}}_{t,j}\right)^\top \left(\mathbf{p}_e - \mathbf{e}\right)_t$$

DiffPD with Dry Frictional Contact

$\boldsymbol{\Gamma}_{\mathbb{C}} = \dfrac{\partial \boldsymbol{\Phi}}{\partial \mathbf{f}}$: is sparse, block diagonal (with one 3 × 3 per contacting node and zeros elsewhere.) and computed per contact in parallel. It is not symmetric in the slip case.

For example imagine we have a system with 5 particles and only particle 2 and 4 have contact, then:

$$\boldsymbol{\Gamma}_{\mathbb{C}} = \begin{bmatrix} \mathbf{0} & & & & \\ & \boldsymbol{\Gamma}_2 & & & \\ & & \mathbf{0} & & \\ & & & \boldsymbol{\Gamma}_4 & \\ & & & & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{3\cdot 5 \times 3\cdot 5} \qquad \boldsymbol{\Gamma}_j = \frac{\partial \boldsymbol{\Phi}_j}{\partial \mathbf{f}_j} \in \mathbb{R}^{3\times 3}$$

	$\mathbf{r}_j$	$\boldsymbol{\Gamma}_j$
take-off	$\mathbf{0}$	$\mathbf{0}$
stick	$-\mathbf{d}_j$	$-\mathbf{I}$
slip	$-d_{\mathrm{N}}\,\hat{\mathbf{n}} + \mu\, d_{\mathrm{N}}\,\hat{\mathbf{d}}_{\mathrm{T}}$	$-\hat{\mathbf{n}}\hat{\mathbf{n}}^{\top} + \mu\,\hat{\mathbf{d}}_{\mathrm{T}}\hat{\mathbf{n}}^{\top} + \mu\,\frac{d_{\mathrm{N}}}{\lVert \mathbf{d}_{\mathrm{T}} \rVert}\,\hat{\mathbf{s}}\hat{\mathbf{s}}^{\top}$

DiffPD with Dry Frictional Contact

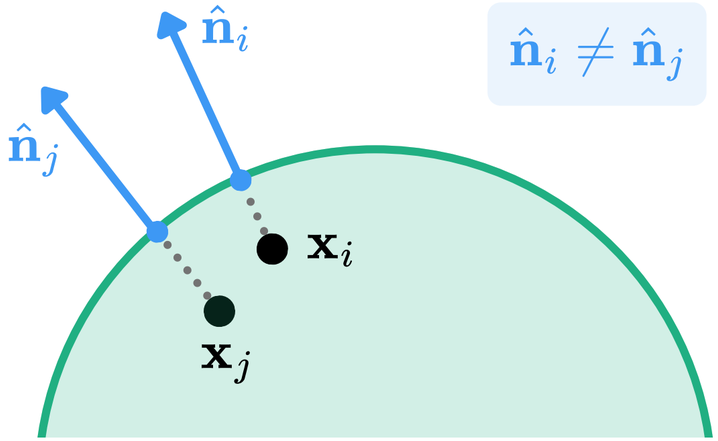
Contacts are detected at the predicted position: $\tilde{\mathbf{x}}_i = \mathbf{x}_i^- + h\,\mathbf{v}_i^-$

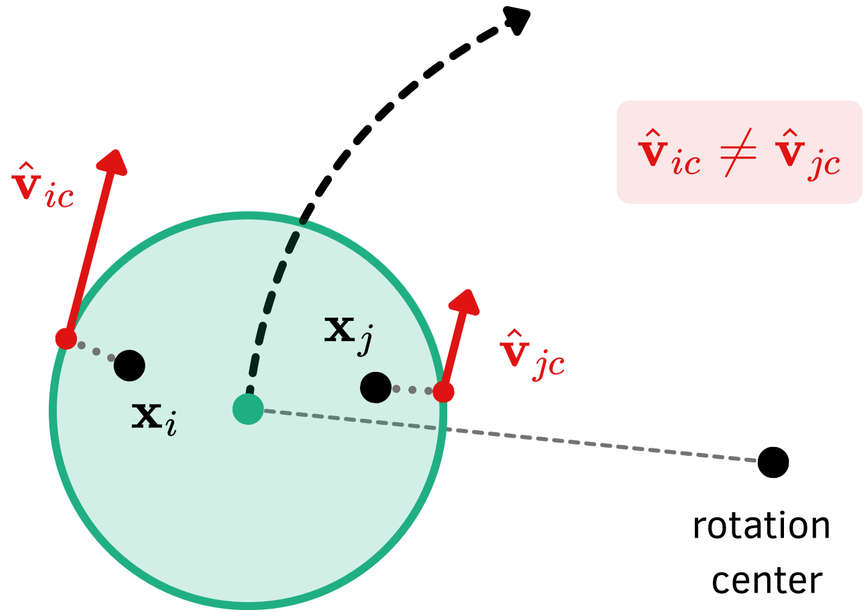
And two contact quantities are evaluated there:

$$\mathbf{x}_i^- \;\rightarrow\; \tilde{\mathbf{x}}_i \;\rightarrow\; \begin{cases} \hat{\mathbf{n}}_i(\tilde{\mathbf{x}}_i) \\ \mathbf{v}_c(\tilde{\mathbf{x}}_i) \end{cases}$$

Contact Normal: changes on a **curved** collider

Collider Velocity: changes on a **rotating** collider





DiffPD with Dry Frictional Contact

Contacts are detected at the predicted position: $\tilde{\mathbf{x}}_i = \mathbf{x}_i^- + h\,\mathbf{v}_i^-$

And two contact quantities are evaluated there:

$$\mathbf{x}_i^- \;\rightarrow\; \tilde{\mathbf{x}}_i \;\rightarrow\; \begin{cases} \hat{\mathbf{n}}_i(\tilde{\mathbf{x}}_i) & \textbf{Contact Normal}\text{: changes on a } \textbf{curved} \text{ collider} \\ \mathbf{v}_c(\tilde{\mathbf{x}}_i) & \textbf{Collider Velocity}\text{: changes on a } \textbf{rotating} \text{ collider} \end{cases}$$

The frictionless impulse at an active contact is: $\mathbf{r}_i = -\hat{\mathbf{n}}_i\hat{\mathbf{n}}_i^\top(\mathbf{f}_i - m_i\,\mathbf{v}_c)$

DiffPD with Dry Frictional Contact

Contacts are detected at the predicted position: $\tilde{\mathbf{x}}_i = \mathbf{x}_i^- + h\,\mathbf{v}_i^-$

And two contact quantities are evaluated there:

$$\mathbf{x}_i^- \;\rightarrow\; \tilde{\mathbf{x}}_i \;\rightarrow\; \begin{cases} \hat{\mathbf{n}}_i\big(\tilde{\mathbf{x}}_i\big) & \textbf{Contact Normal}\text{: changes on a } \textbf{curved} \text{ collider} \\ \mathbf{v}_c\big(\tilde{\mathbf{x}}_i\big) & \textbf{Collider Velocity}\text{: changes on a } \textbf{rotating} \text{ collider} \end{cases}$$

The frictionless impulse at an active contact is: $\mathbf{r}_i = -\hat{\mathbf{n}}_i\hat{\mathbf{n}}_i^\top\big(\mathbf{f}_i - m_i\,\mathbf{v}_c\big)$

$$\frac{\partial\,\mathbf{r}_i}{\partial\,\mathbf{x}_i^-} = \underbrace{-\,\hat{\mathbf{n}}_i\hat{\mathbf{n}}_i^\top\,\frac{\partial\,\mathbf{f}_i}{\partial\,\mathbf{x}_i^-}}_{\textbf{Already Counted}} \underbrace{-\,\frac{\partial\,(\hat{\mathbf{n}}_i\hat{\mathbf{n}}_i^\top)}{\partial\,\mathbf{x}_i^-}\,\big(\mathbf{f}_i - m_i\,\mathbf{v}_c\big)}_{\textbf{Curvature Correction} = \Delta_{\mathrm{curv}}} \underbrace{+\,m_i\,\hat{\mathbf{n}}_i\hat{\mathbf{n}}_i^\top\,\frac{\partial\,\mathbf{v}_c}{\partial\,\mathbf{x}_i^-}}_{\textbf{Rotation Correction} = \Delta_{\mathrm{rot}}}$$

DiffPD with Dry Frictional Contact

Contacts are detected at the predicted position: $\tilde{\mathbf{x}}_i = \mathbf{x}_i^- + h\,\mathbf{v}_i^-$

And two contact quantities are evaluated there:

$$\mathbf{x}_i^- \;\rightarrow\; \tilde{\mathbf{x}}_i \;\rightarrow\; \begin{cases} \hat{\mathbf{n}}_i\big(\tilde{\mathbf{x}}_i\big) & \textbf{Contact Normal}\text{: changes on a } \textbf{curved} \text{ collider} \\ \mathbf{v}_c\big(\tilde{\mathbf{x}}_i\big) & \textbf{Collider Velocity}\text{: changes on a } \textbf{rotating} \text{ collider} \end{cases}$$

The frictionless impulse at an active contact is: $\mathbf{r}_i = -\hat{\mathbf{n}}_i\hat{\mathbf{n}}_i^\top\big(\mathbf{f}_i - m_i\,\mathbf{v}_c\big)$

$$\frac{\partial\,\mathbf{r}_i}{\partial\,\mathbf{x}_i^-} = \underbrace{-\,\hat{\mathbf{n}}_i\hat{\mathbf{n}}_i^\top\,\frac{\partial\,\mathbf{f}_i}{\partial\,\mathbf{x}_i^-}}_{\textbf{Already Counted}} \;\underbrace{-\;\frac{\partial\,(\hat{\mathbf{n}}_i\hat{\mathbf{n}}_i^\top)}{\partial\,\mathbf{x}_i^-}\,\big(\mathbf{f}_i - m_i\,\mathbf{v}_c\big)}_{\textbf{Curvature Correction} = \Delta_{\mathrm{curv}}} \;\underbrace{+\;m_i\,\hat{\mathbf{n}}_i\hat{\mathbf{n}}_i^\top\,\frac{\partial\,\mathbf{v}_c}{\partial\,\mathbf{x}_i^-}}_{\textbf{Rotation Correction} = \Delta_{\mathrm{rot}}}$$

The gradient is linear in this derivative so each new term simply adds on top of the base formula:

$$\frac{d\,\phi}{d\,\mathbf{x}_i^-} = \Big[\bar{\mathbf{z}}^\top\Big(h\,\Delta\mathbf{L} - \tfrac{1}{h}\mathbf{C}\Big) + \frac{\partial\,\phi}{\partial\,\mathbf{x}^+}\Big]_i + \Delta_{\mathrm{curv}} + \Delta_{\mathrm{rot}}$$

DiffPD with Dry Frictional Contact

- **Curvature Correction**:

$$\Delta_{\text{curv}} = -\,\mathbf{S}_i \Big[\, d_{\text{N}}\,\mathbf{z}_i + (\hat{\mathbf{n}}_i \cdot \mathbf{z}_i)\, m_i \big(\mathbf{v}_i^{+} - \mathbf{v}_c\big)\Big]$$

plane	$\mathbf{S}_i = \mathbf{0}$
sphere	$\mathbf{S}_i = \frac{1}{\rho_i}\big(\mathbf{I} - \hat{\mathbf{n}}_i \hat{\mathbf{n}}_i^{\top}\big)$
cylinder	$\mathbf{S}_i = \frac{1}{\rho_i}\big(\mathbf{I} - \hat{\mathbf{n}}_i \hat{\mathbf{n}}_i^{\top} - \mathbf{a}\mathbf{a}^{\top}\big)$

- **Rotation Correction**:

$$\Delta_{\text{rot}} = -\,m_i\,(\hat{\mathbf{n}}_i \cdot \mathbf{z}_i) \left(\frac{\partial\, \mathbf{g}_i}{\partial\, \tilde{\mathbf{x}}_i}\right)^{\top} \big(\boldsymbol{\omega} \times \hat{\mathbf{n}}_i\big)$$

	$\frac{\partial\, \mathbf{g}_i}{\partial\, \tilde{\mathbf{x}}_i}$	Δ_{rot}
sphere	$r\,\mathbf{S}_i$	$-\frac{m_i r}{\rho_i}\,(\hat{\mathbf{n}}_i \cdot \mathbf{z}_i)\,\big(\boldsymbol{\omega} \times \hat{\mathbf{n}}_i\big)$
cylinder	$\mathbf{a}\mathbf{a}^{\top} + r\,\mathbf{S}_i$	$-m_i\,(\hat{\mathbf{n}}_i \cdot \mathbf{z}_i)\,\Big[\mathbf{c}_{\parallel} + \frac{r}{\rho_i}\,\mathbf{c}_{\perp}\Big],\quad \mathbf{c} = \boldsymbol{\omega} \times \hat{\mathbf{n}}_i$

DiffPD with Dry Frictional Contact

function DIFFPDCONTACTBACKWARDSTEP($\mathbf{x}^+, \mathbf{v}^+, \frac{\partial \phi}{\partial \mathbf{x}^+}, \frac{\partial \phi}{\partial \mathbf{v}^+}$)

for each element e **in parallel do**

  $\boldsymbol{\Gamma}_e \leftarrow \frac{\partial \mathbf{p}_e}{\partial (\mathbf{G}_e \mathbf{x})}$

$\mathbf{f} \leftarrow \tilde{\mathbf{b}} - \mathbf{C}\,\mathbf{v}^+$

for each contact i **in parallel do**

  $d_{\mathrm{N},i} \leftarrow \hat{\mathbf{n}}_i^\top \left(\mathbf{f}_i - m_i\,\mathbf{v}_c\right)$ active if $d_{\mathrm{N},i} < 0$

$\mathbf{P} \leftarrow \text{blockdiag}\left(\hat{\mathbf{n}}_i \hat{\mathbf{n}}_i^\top\right)$ over active contacts

$\mathbf{s} \leftarrow \left(\frac{\partial \phi}{\partial \mathbf{v}^+} + h\,\frac{\partial \phi}{\partial \mathbf{x}^+}\right)^\top$

Initialize $\mathbf{z}$

for $k = 0, 1, \ldots, K-1$ **do**

  $\bar{\mathbf{z}} \leftarrow (\mathbf{I} - \mathbf{P})\,\mathbf{z}$

  $\mathbf{q} \leftarrow h^2\,\Delta\mathbf{L}\,\bar{\mathbf{z}} + \mathbf{C}\,\mathbf{P}\,\mathbf{z}$

  SOLVE $\mathbf{L}\,\mathbf{z} = \mathbf{q} + \mathbf{s}$

$\frac{\partial \phi}{\partial \mathbf{v}^-} \leftarrow \bar{\mathbf{z}}^\top \mathbf{M}$

$\frac{\partial \phi}{\partial \mathbf{x}^-} \leftarrow \bar{\mathbf{z}}^\top \left(h\,\Delta\mathbf{L} - \frac{1}{h}\mathbf{C}\right) + \frac{\partial \phi}{\partial \mathbf{x}^+}$

for each active contact i **in parallel do**

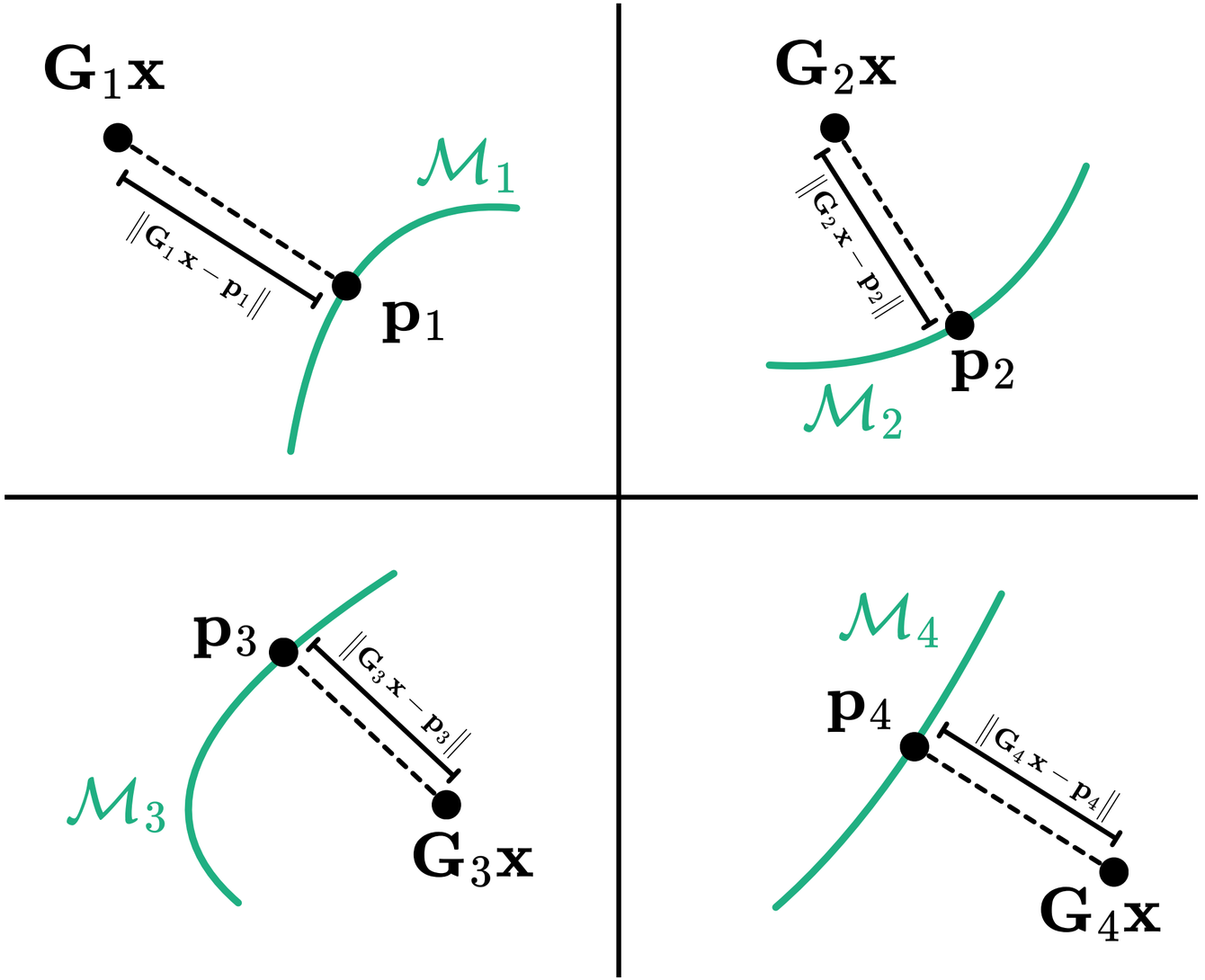
  $\Delta_i \leftarrow \Delta_{\text{curv}} + \Delta_{\text{rot}}$

  $\left(\frac{\partial \phi}{\partial \mathbf{x}^-}\right)_i \mathrel{+}= \Delta_i, \quad \left(\frac{\partial \phi}{\partial \mathbf{v}^-}\right)_i \mathrel{+}= h\,\Delta_i$

Newton-Raphson Method Single Iteration

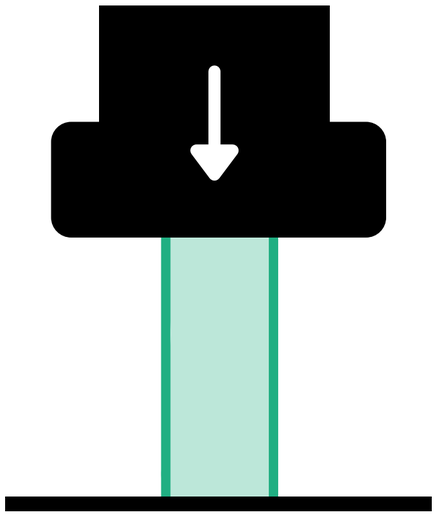
$$\left[\mathbf{M} - h\,\frac{\partial\,\mathbf{f}}{\partial\,\mathbf{v}} - h^2\,\frac{\partial\,\mathbf{f}}{\partial\,\mathbf{x}}\right]\Delta\mathbf{x} = h\left[h\,\mathbf{f}(\mathbf{x}^-, \mathbf{v}^-) + \left(\mathbf{M} - h\,\frac{\partial\,\mathbf{f}}{\partial\,\mathbf{v}}\right)\mathbf{v}^-\right]$$

Projective Dynamics (PD)

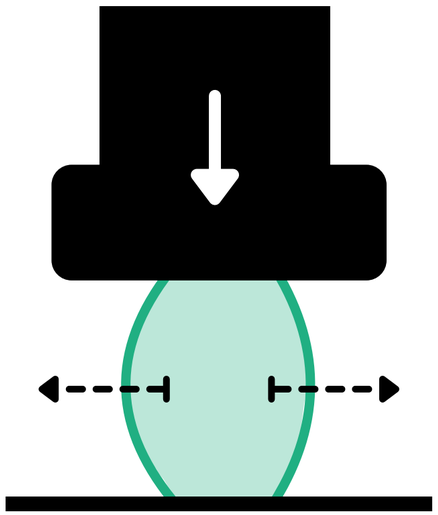


Multiple Valid Energy States

t_1



t_2



t_3

